

Relatório de Estágio: Avaliação de Plataformas de Gestão de Dispositivos ESP32

Francisco José Nogueira da Rocha Matos

7 de agosto de 2026

Conteúdo

1	Introdução	3
2	Configuração do Ambiente de Desenvolvimento	3
3	Camada de Firmware 1: Zephyr RTOS	3
3.1	Validação de Conectividade	3
3.2	Desafios Encontrados	3
3.3	Atualização Remota de Firmware (OTA)	4
3.3.1	MCUboot + mcumgr (transporte série)	4
3.3.2	LwM2M (transporte Wi-Fi)	4
3.4	Diagnóstico Complementar	5
3.5	Resolução: Causa Raiz Confirmada e Corrigida	5
3.6	Suporte a Drivers de Sensores I2C e Comunicação LoRa	6
4	Camada de Firmware 2: ESP-IDF (nativo)	7
4.1	Mecanismo de OTA	7
4.2	Resultado: Atualização Bem-Sucedida	7
4.3	Esforço de Integração	8
4.4	Resultado: <i>Rollback</i> Automático Confirmado	8
4.5	Suporte a Drivers de Sensores I2C e Comunicação LoRa	8
5	Plataforma de Gestão 1: ThingsBoard	9
5.1	Ligação e Telemetria	9
5.2	Configuração Remota (<i>Shared Attributes</i>)	9
5.3	RPC (<i>Remote Procedure Call</i>)	9
5.4	Atualização Remota de Firmware (OTA)	9
5.5	<i>Rollback</i> Automático e Proteção Contra Ciclos de Repetição	10
5.6	Alarmes e Notificações	10
5.7	Provisionamento Automático de Dispositivos	11
5.8	Aplicação Móvel	11
5.9	Esforço de Integração	11
5.10	Segurança do Transporte (MQTT/TLS)	11
5.11	Prova de Conceito: Cliente Nativo Zephyr	11

6	Plataforma de Gestão 2: ESP RainMaker	12
6.1	Esforço de Integração Inicial	12
6.2	Arquitetura de Provisionamento – Diferença Relevante	13
6.3	Resultado: Acompanhamento via Consola Web Confirmado	13
6.4	Atualização Remota de Firmware (OTA)	13
6.5	Comandos Remotos (RPC)	14
6.6	Funcionalidades de Gestão de Frota (levantamento geral)	14
6.7	Achado Espontâneo: Notificação de Desconexão Sem Configuração	15
6.8	Reavaliação com Foco em Gestão do Dispositivo	15
6.9	Síntese da Comparação com o ThingsBoard	17
7	Plataforma de Gestão 3: Azure IoT Hub + Device Provisioning Service (DPS)	17
7.1	Configuração dos Recursos na Azure	17
7.2	Arquitetura de Provisionamento	17
7.3	Esforço de Integração (Firmware)	18
7.4	Resultado: Registo e Telemetria Validados	18
7.5	Comandos Síncronos: Direct Methods	19
7.6	Configuração Remota: Device Twin	19
7.7	Reorientação de Foco: Gestão do Dispositivo, não Telemetria	19
7.8	Mensagens Cloud-to-Device (C2D)	20
7.9	Atualização Remota de Firmware (OTA)	20
7.10	Alarmes/Notificações: Azure Monitor (explorado, não validado)	21
7.11	Message Routing (exploração lateral – dados, não dispositivo)	22
7.12	Gestão de Frota: Configurations + Deployments	23
7.13	Provisionamento em Escala: Group Enrollment	23
7.14	Limitações do Âmbito deste PoC	24
8	Síntese Comparativa	24
8.1	Mecanismos de OTA (dispositivo)	24
8.2	Plataformas de Gestão de Dispositivos (visão mais ampla)	24
8.3	Ver vs. Gerir: Quem Implementa Cada Capacidade	24
8.4	Integração de Drivers de Sensores I2C e LoRa: Zephyr vs. ESP-IDF	25
8.5	Prova de Conceito: Driver MCP3221 (Zephyr vs. ESP-IDF)	26
8.6	Prova de Conceito: Driver RN2483 (LoRa)	26
9	Candidato Adicional: Mongoose OS	27
9.1	Estado de Manutenção	27
9.2	O Que Substituiu o Mongoose OS	27
9.3	Teste Prático: Instalação, Compilação e Gravação	28
9.4	Drivers de Sensores e LoRa	29
9.5	Funcionalidades Notáveis (mesmo não adotado)	29
10	Trabalho Futuro	30
11	Conclusão Preliminar	31

1 Introdução

Este documento descreve as atividades realizadas durante um estágio de 1 mês na Watgrid, com o objetivo de avaliar plataformas e sistemas operativos para o hardware ESP32 e identificar quais melhor suportam o caso de uso real da empresa. O foco inicial foi a capacidade de atualização remota de firmware (OTA – *Over-The-Air*); ao longo do estágio, esse foco foi alargado à facilidade de integração de *drivers* de sensores I2C e de comunicação LoRa, refletindo a evolução dos critérios de avaliação registada ao longo deste relatório. Foram avaliadas seis camadas complementares: o RTOS Zephyr, o SDK nativo ESP-IDF da Espressif e o Mongoose OS (todos ao nível do firmware que corre no próprio dispositivo), e três plataformas de gestão de dispositivos na nuvem (*device management*) – ThingsBoard, ESP RainMaker e Azure IoT Hub + Device Provisioning Service –, avaliadas como forma de analisar o problema também a um nível mais abrangente que o RTOS/SDK isolado.

2 Configuração do Ambiente de Desenvolvimento

Foi estabelecido um ambiente de trabalho utilizando o VS Code. As principais ações incluíram:

- Configuração de tarefas automatizadas (`tasks.json`) para compilação (`west build`) e gravação (`west flash`) de firmware, no caso do Zephyr.
- Gestão do ambiente virtual Python (`zephyr-env`) para isolamento de dependências.
- Ativação de um ambiente ESP-IDF (v5.5.1) já instalado, para a segunda plataforma avaliada.

3 Camada de Firmware 1: Zephyr RTOS

3.1 Validação de Conectividade

O objetivo desta fase foi demonstrar a capacidade de um dispositivo ESP32, a correr Zephyr, integrar-se numa rede sem fios e obter conectividade IP.

A configuração da stack de rede foi realizada via `prj.conf`, ativando `CONFIG_NETWORKING` e `CONFIG_NET_SHELL`. A validação foi realizada via *Shell* (UART), utilizando os comandos `wifi connect -s "SSID-p "PASS-k 1` (ligação WPA2-PSK) e `net iface` (verificação de endereço DHCP).

3.2 Desafios Encontrados

- **Gestão de Binários:** Necessidade de importar bibliotecas de RF (*blobs*) da Espressif.
- **Configuração de Segurança Wi-Fi:** Necessidade de especificar explicitamente o método de gestão de chaves (`-k 1`).

- **Ligação automática no arranque:** esta versão do Zephyr (3.7.0) não possui subsistema de credenciais Wi-Fi persistentes. Foi implementada uma ligação automática programática (`net_mgmt` com `NET_REQUEST_WIFI_CONNECT`) para evitar a necessidade de reconfiguração manual a cada reinício.
- **Bug de deteção silenciosa:** a opção `CONFIG_NET_CONNECTION_MANAGER`, necessária para que a aplicação aguardasse corretamente pela disponibilidade de rede antes de iniciar operações, não estava ativa. Por se tratar de uma verificação em tempo de compilação (`IS_ENABLED`), o compilador eliminava silenciosamente o bloco de código relevante, sem qualquer aviso, causando comportamento incorreto de difícil diagnóstico.

3.3 Atualização Remota de Firmware (OTA)

Foram avaliadas duas arquiteturas distintas de OTA, ambas assentes no mecanismo de *dual-slot* do MCUboot: a imagem de firmware corrente ocupa uma partição primária, enquanto uma nova imagem é escrita numa partição secundária; no arranque seguinte, o MCUboot decide se aplica a atualização pendente.

3.3.1 MCUboot + mcumgr (transporte série)

Nesta abordagem, o `mcumgr` foi utilizado como protocolo de transporte para envio da imagem de firmware através da porta série, complementando o MCUboot, que gere a decisão de arranque entre partições.

Resultado: o ciclo completo de atualização (envio da imagem, confirmação, reinício) foi validado com sucesso.

Limitação identificada: a configuração de fábrica da Espressif para o MCUboot no ESP32 opera em modo *upgrade-only* – a atualização é aplicada de forma direta e permanente, sem fase de teste prévio, sem mecanismo de *rollback* automático. Uma tentativa de ativar o modo de *swap* com possibilidade de reversão (*swap-using-scratch*) resultou em falha crítica do *bootloader* (ciclo de reinício).

Observação relevante ao caso de uso: este mecanismo depende de ligação por cabo série, não refletindo o cenário de operação real de um dispositivo em campo.

3.3.2 LwM2M (transporte Wi-Fi)

Nesta abordagem, foi implementado um cliente LwM2M (*Lightweight Machine-to-Machine*) sobre Wi-Fi, com o servidor Leshan (*Eclipse Foundation*) a correr localmente. O protocolo LwM2M expõe um objeto *standard* dedicado a atualizações de firmware (*Object 5 – Firmware Update*), através do qual a imagem é transferida por blocos via CoAP e escrita na partição secundária.

Resultados obtidos: ligação Wi-Fi automática e registo estável do cliente no servidor Leshan validados. Receção e escrita parcial de blocos de firmware confirmadas nos registos, validando a correção da implementação do mecanismo de receção.

Falha crítica identificada: o dispositivo apresenta uma exceção fatal (*illegal instruction*) de forma consistente durante a escrita da imagem na memória flash, sempre que esta ocorre em simultâneo com atividade da interface Wi-Fi. A hipótese técnica é de contenção no acesso à *cache* de instruções do ESP32, da qual o próprio controlador Wi-Fi depende para execução de código. Foram tentadas três mitigações sucessivas (*buffers*

de rede maiores, *stack* maior, bloqueio de interrupções durante a escrita), sem resolução completa. A atualização nunca foi concluída com sucesso nos testes realizados nesta plataforma.

3.4 Diagnóstico Complementar

Antes de se concluir tratar-se de uma limitação da plataforma, foram eliminadas hipóteses alternativas: incompatibilidade de segurança Wi-Fi (confirmado WPA2, não WPA3, via inspeção direta do ponto de acesso), MTU anômalo (confirmado valor padrão de 1500 bytes) e configuração de *clock* da flash (valor padrão de 40MHz, sem erros reportados). Adicionalmente, a falha ocorria sempre exatamente no mesmo ponto lógico (fronteiras de 512 bytes no buffer interno de escrita), um padrão de repetibilidade típico de uma condição de corrida em software, não de instabilidade de rede.

Como teste adicional, o mesmo cenário foi repetido numa segunda rede Wi-Fi, com significativamente menos tráfego de fundo. Com a mitigação de bloqueio de interrupções (`irq_lock()`) ainda ativa, a transferência progrediu consideravelmente mais (aproximadamente 25% da imagem) sem exceção fatal, mas ficou permanentemente bloqueada sem concluir – efeito secundário do próprio bloqueio de interrupções, que atrasa a confirmação de receção ao ponto de causar retransmissões CoAP espúrias. Removendo essa mitigação e repetindo o teste na mesma rede de baixo tráfego, a exceção fatal reapareceu de forma imediata e determinística, na primeira escrita real em flash, no mesmo endereço de instrução dos testes anteriores. Este resultado confirma de forma conclusiva que a causa da falha é independente da rede utilizada, sendo unicamente determinada pela ocorrência simultânea de escrita em flash e atividade da interface Wi-Fi – encerrando esta linha de diagnóstico.

3.5 Resolução: Causa Raiz Confirmada e Corrigida

Numa fase posterior de aprofundamento do firmware Zephyr, esta falha foi revisitada e efetivamente resolvida.

Causa raiz: pesquisa no rastreador de problemas oficial do Zephyr (`zephyrproject-rtos/zephyr`) encontrou a *issue* #77952, que descreve exatamente o sintoma observado: o mecanismo que desativa interrupções cujo código reside em flash durante uma operação de escrita (`esp_intr_noniram_disable()`) funcionava corretamente na versão 3.6 do Zephyr, mas deixou de ser invocado na 3.7.0 – uma regressão confirmada pelo próprio mantenedor Espressif do projeto. Sem esta proteção, uma interrupção de Wi-Fi cujo código não esteja inteiramente em IRAM pode disparar durante a janela em que a *cache* de flash está desativada para a escrita, causando exatamente a exceção *illegal instruction* observada.

Correção aplicada: a *issue* tem uma correção oficial já integrada (*PR* #78121, *merged* para o `v3.7-branch`), que reordena a sequência de arranque em `soc/espressif/esp32/soc.c` e atualiza o módulo `hal_espressif` para uma revisão mais recente. O checkout local usado neste estágio é anterior a esta correção (de julho de 2024; a correção é de setembro de 2024), pelo que a alteração foi replicada manualmente: atualização do módulo `hal_espressif` para o *commit* corrigido e aplicação do mesmo *patch* ao ficheiro `soc.c`.

Validação com hardware real: removida também a mitigação manual (`irq_lock()/irq_unlock()`) do código do projeto, para testar a correção oficial isoladamente, sem rede de segurança própria. Repetindo exatamente o cenário que antes falhava de forma determinística (envio do firmware completo por LwM2M/CoAP sobre Wi-Fi, com a interface ativa durante

toda a escrita): a imagem completa (680676 bytes, 593 blocos) foi recebida e escrita **sem qualquer exceção**. Acionado de seguida o recurso "Update" do Object 5, o dispositivo reiniciou de forma limpa (*reset* de software, não uma exceção), o MCUboot confirmou a aplicação permanente da nova imagem ("*Swap type: perm*"), e o dispositivo voltou a registar-se no Leshan com sucesso.

Achado corroborante: durante esta investigação, foi encontrada uma segunda *issue*, aberta na véspera deste teste e ainda sem resolução (#114394), que reproduz o mesmo padrão de falha – escrita em flash a coincidir com atividade de um controlador de rádio, resultando em *illegal instruction* – desta vez com Bluetooth em vez de Wi-Fi, e no ESP32-C3 em vez do ESP32 original. Isto indica que a falha encontrada não é uma particularidade isolada deste projeto, mas sintoma de uma classe de problema mais ampla na integração entre operações de flash e controladores de rádio nos SoCs Espressif suportados pelo Zephyr.

Implicação: a limitação mais grave identificada nesta avaliação do Zephyr – a impossibilidade de concluir uma atualização de firmware sem fios – está resolvida. Este resultado é particularmente relevante à luz do requisito de suporte nativo a LoRa (secção 3.6): a razão principal que anteriormente pesava contra o Zephyr deixa de se aplicar.

3.6 Suporte a Drivers de Sensores I2C e Comunicação LoRa

Um critério adicional de avaliação do firmware é a facilidade de integração de *drivers* de sensores I2C e de comunicação LoRa, relevante para o tipo de sensores usados em monitorização de vinha (ADC, acelerómetros) e para o alcance necessário em campo aberto. Foi investigado o suporte nativo (*in-tree*) do Zephyr 3.7.0 para três sensores concretos – MCP3221 (ADC I2C, Microchip), KX122 (acelerómetro, ROHM/Kionix) e IIS2ICLX (acelerómetro/inclinómetro, ST) – e para comunicação LoRa.

- **IIS2ICLX:** *driver* nativo completo (`drivers/sensor/st/iis2iclx/`), com suporte a barramento I2C e SPI, modo de *trigger* (interrupções) e *sensor hub*.
- **KX122:** não encontrado nos *drivers in-tree* do Zephyr.
- **MCP3221:** não encontrado nos *drivers in-tree*. Existe suporte nativo para a família MCP320x (`drivers/adc/adc_mcp320x.c`), mas trata-se de um ADC diferente, com interface SPI em vez de I2C.
- **LoRa:** suporte nativo e completo (`drivers/lora/`), cobrindo os transdutores Semtech SX126x e SX127x, módulos com comandos AT (RYLR), e uma API unificada (`lora_send()`/`lora_recv()`) mantida como parte da árvore do próprio SO.

O Zephyr trata sensores e LoRa como parte do subsistema de *drivers* do próprio sistema operativo – mesmo quando um chip específico não está coberto (KX122, MCP3221), a integração de um novo *driver* segue um padrão e uma API já estabelecidos e documentados.

Nota importante sobre módulos LoRa concretos (RN2483, WLR089U0): o suporte nativo acima (`drivers/lora/`) é para transdutores "*nus*" (Semtech SX126x/SX127x), controlados diretamente por SPI. Os módulos RN2483 e WLR089U0 da Microchip – candidatos concretos considerados para este projeto – são diferentes: têm o seu próprio microcontrolador embutido e comunicam com o hospedeiro por UART, através de um conjunto de comandos de texto próprio da Microchip, não por SPI direto ao rádio. Não

foi encontrado suporte nativo a nenhum dos dois no Zephyr (nem no `drivers/lora/`, nem em nenhum outro módulo do ecossistema West). O WLR089U0 assenta no chip SAM R34, para o qual o Zephyr tem suporte de placa (`samr34_xpro`) – mas isso implica usar o SAM R34 como microcontrolador principal (substituindo o ESP32), um cenário de arquitetura diferente do de um módulo LoRa satélite, e não foi confirmado se essa placa já tem o *driver* do rádio ligado.

Esta lacuna não é, no entanto, equivalente a um *driver* de rádio em falta: por ser um módulo com comandos de texto, integrar o RN2483 resume-se a escrever um *wrapper* sobre UART (montar strings de comando, interpretar respostas), tarefa de escala semelhante à do MCP3221 (secção 8.5), não à complexidade de um *driver* SPI de baixo nível. Existem ainda bibliotecas de terceiros já escritas especificamente para o RN2483 que podem servir de referência para adaptar – incluindo uma implementação explicitamente independente de plataforma, em C99 puro (`GearSiX/RN2483-Library`, sem manutenção ativa e desatualizada face à especificação LoRaWAN atual, mas didaticamente útil), além de várias bibliotecas Arduino maduras (ex: `jpmeijers/RN2483-Arduino-Library`). Esta afirmação foi validada com código real, não só em teoria – ver a prova de conceito da secção 8.6.

4 Camada de Firmware 2: ESP-IDF (nativo)

Dada a limitação encontrada na secção anterior, foi avaliado o SDK nativo da Espressif (ESP-IDF v5.5.1) como segunda plataforma, com base no exemplo oficial `native_ota_example`.

4.1 Mecanismo de OTA

Ao contrário do LwM2M (modelo *push*), o ESP-IDF utiliza um modelo *pull*: o dispositivo descarrega ativamente a imagem de um servidor HTTP/HTTPS. O esquema de partições é mais robusto que o do MCUboot, com três partições de aplicação (`factory`, `ota_0`, `ota_1`) e uma partição dedicada (`otadata`) que regista qual deve arrancar a seguir. O mecanismo inclui controlo de versão nativo (previne ciclos de atualização infinitos) e suporte documentado a *rollback* automático (`CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`), com uma janela de diagnóstico no primeiro arranque de uma imagem nova.

4.2 Resultado: Atualização Bem-Sucedida

Utilizando um servidor HTTP local e a mesma rede Wi-Fi (Watgrid Guest) da secção anterior, a atualização completa (aproximadamente 915KB) foi transferida e escrita com sucesso, **sem qualquer falha**, com a interface Wi-Fi ativa durante toda a transferência – o mesmo cenário que causava falha consistente na plataforma Zephyr. O dispositivo reiniciou de forma limpa e arrancou com a nova versão do firmware, confirmada por *hash* SHA-256 distinto.

Implicação: este resultado, obtido com o mesmo hardware e a mesma rede da secção 3.3.2, constitui evidência forte de que a falha anteriormente identificada não resulta de uma limitação do módulo ESP32 nem da rede local, mas sim de uma lacuna específica na implementação/coordenação entre escrita de flash e o controlador Wi-Fi no *port* do Zephyr para este hardware.

4.3 Esforço de Integração

O ambiente já se encontrava instalado, exigindo apenas a resolução de um conflito de versão do pacote `setuptools`. O exemplo compilou, foi gravado e executou corretamente, sem as dificuldades de configuração de `devicetree`/Kconfig observadas na plataforma Zephyr.

4.4 Resultado: *Rollback* Automático Confirmado

Foi ativada a opção de *bootloader* `CONFIG_BOOTLOADER_APP_ROLLBACK_ENABLE`, que exige uma gravação inicial por cabo (as atualizações subsequentes continuam a ser feitas por OTA). Para validar o mecanismo, foi publicada via OTA uma imagem deliberadamente defeituosa (com uma chamada a `abort()` no arranque da aplicação, simulando uma atualização malformada). O dispositivo descarregou a imagem, arrancou, falhou imediatamente e, sem qualquer intervenção humana, reverteu automaticamente para a última versão estável no arranque seguinte. Os registos confirmam ainda uma proteção contra ciclos de repetição: o *bootloader* regista a versão que falhou e recusa-se a tentar aplicá-la novamente de forma automática. Este resultado contrasta diretamente com a tentativa equivalente no MCUboot/Zephyr (secção 3.3.1), onde a ativação de um modo de *swap* com reversão resultou em falha do próprio *bootloader*, nunca chegando a ser validada.

4.5 Suporte a Drivers de Sensores I2C e Comunicação LoRa

Repetindo a mesma investigação da secção 3.6 para o ESP-IDF v5.5.1: ao contrário do Zephyr, o repositório principal do ESP-IDF não inclui um subsistema de *drivers* de sensores – os componentes nativos limitam-se a periféricos genéricos do próprio microcontrolador (`esp_driver_i2c`, `esp_driver_spi`, `esp_adc` para o ADC interno). Não existe, em nenhum caso, uma pasta equivalente à `drivers/sensor/` do Zephyr.

Consultado o *ESP Component Registry* (`components.espressif.com`), repositório oficial de componentes de terceiros para ESP-IDF:

- **IIS2ICLX, KX122 e MCP3221:** nenhum dos três tem um componente publicado no registo.
- **MCP3221:** apesar de ausente do registo, é um ADC muito simples de integrar de raiz – uma única leitura I2C de 2 bytes, sem registos de configuração – pelo que a ausência de biblioteca pronta tem impacto prático reduzido.
- **LoRa:** sem suporte nativo, mas com componentes de terceiros maduros e ativamente mantidos disponíveis no registo (`jgromes/radiolib`, `dernasherebrezon/sx127x`), para além de várias implementações adicionais disponíveis diretamente no GitHub.

Ao contrário do Zephyr, o ESP-IDF depende sempre de bibliotecas de terceiros para *drivers* de sensores e LoRa – funcional na prática, mas sem o mesmo nível de suporte/manutenção oficial da Espressif que os componentes do core (Wi-Fi, OTA, etc.) têm.

Nota sobre módulos LoRa concretos (RN2483, WLR089U0): tal como no Zephyr (secção 3.6), as bibliotecas de terceiros encontradas (`radiolib`, `sx127x`) são para transdutores SPI *nus*, não para estes módulos com comando por texto via UART – confirmado que o RadioLib não suporta o RN2483. Não foi encontrado nenhum componente para o RN2483 ou o WLR089U0 no ESP Component Registry. A situação é, portanto, equivalente à do Zephyr para este caso concreto: nenhum dos dois *frameworks* tem isto

pronto, e em ambos a integração resume-se a um *wrapper* de comandos por UART (ver bibliotecas de referência citadas na secção 3.6).

5 Plataforma de Gestão 1: ThingsBoard

Ao contrário do Zephyr e do ESP-IDF, que constituem alternativas diretas para o firmware que corre *dentro* do ESP32, o ThingsBoard é uma plataforma de *device management* que corre na nuvem: não substitui o SDK do dispositivo, complementa-o, exigindo sempre um dos dois anteriores por baixo (utilizou-se o ESP-IDF para a avaliação principal, dado o seu desempenho superior nas secções anteriores; o Zephyr foi validado à parte, secção 5.11, confirmando que também funciona). Foi avaliada a oferta gratuita *ThingsBoard Cloud* (*Free tier*, sem necessidade de cartão de crédito, limitada a 5 dispositivos), como alternativa de menor fricção face a plataformas equivalentes como o AWS IoT Core ou o Azure IoT Hub – ambas avaliadas apenas por documentação, dado exigirem criação de conta com cartão de crédito e um esforço de integração consideravelmente maior (certificados X.509, políticas IAM).

5.1 Ligação e Telemetria

O dispositivo liga-se por MQTT, autenticando-se com um *access token* simples como *username* (sem certificados, ao contrário do esquema X.509 típico da AWS). A validação inicial foi feita sem hardware, usando um cliente MQTT de linha de comandos para confirmar o *endpoint* e o token antes de qualquer alteração ao firmware. O envio de telemetria simulada (*temperature*) funcionou de forma imediata e estável, sem os obstáculos de configuração (Kconfig, *devicetree*) observados no Zephyr.

5.2 Configuração Remota (*Shared Attributes*)

O ThingsBoard permite definir valores de configuração (*shared attributes*) que o servidor "empurra" para o dispositivo via MQTT, sem necessidade de *reflash*. Foi implementado e testado com sucesso o controlo remoto do intervalo de publicação de telemetria (*publishInterval*): uma alteração feita na consola refletiu-se no comportamento do dispositivo em tempo real, confirmado pelos registos série.

5.3 RPC (*Remote Procedure Call*)

Complementarmente aos *shared attributes* (configuração persistida, assíncrona), foi implementado suporte a RPC de duas vias – um pedido síncrono com resposta imediata. Foram implementados dois métodos de exemplo, *getStatus* (devolve telemetria e memória livre atuais) e *reboot*. Testado via API REST do ThingsBoard (não existe um botão de envio de RPC simples na interface de dispositivo, ao contrário dos *attributes*), com resposta correta e imediata do dispositivo.

5.4 Atualização Remota de Firmware (OTA)

O ThingsBoard implementa um mecanismo de OTA próprio sobre MQTT, distinto tanto do modelo *push* por blocos CoAP do LwM2M como do modelo *pull* HTTP contínuo do ESP-IDF nativo: o dispositivo pede blocos (*chunks*) de firmware um a um (`v2/fw/request/{id}/chunk/`

controlando o próprio ritmo do download, com verificação de integridade por *checksum* (SHA-256) calculado incrementalmente à medida que os blocos chegam, sem necessidade de armazenar o binário completo em RAM. O progresso é reportado à plataforma através de uma máquina de estados (DOWNLOADING → DOWNLOADED → VERIFIED → UPDATING → UPDATED), visível em tempo real na consola.

Resultado: atualização completa e sem falhas, reaproveitando o esquema de partições de duplo slot já validado no ESP-IDF nativo. Foi identificado e corrigido um problema de coerência de versão – o firmware reportava uma versão fixa no código, nunca incrementada entre compilações, causando um ciclo de atualização infinito (o dispositivo nunca reconhecia a versão nova como sua) – resolvido ao garantir que a versão reportada pelo dispositivo é sempre coerente com a publicada no repositório de OTA antes de cada compilação.

5.5 Rollback Automático e Proteção Contra Ciclos de Repetição

Ativado o mesmo mecanismo de *rollback* do *bootloader* já validado no ESP-IDF nativo (secção 4.4), reutilizando a infraestrutura de partições duplas. Repetido o teste de falha real (imagem com `abort()` deliberado publicada como atualização): a placa descarregou a imagem defeituosa, reiniciou, falhou imediatamente e o *bootloader* reverteu automaticamente para a última versão estável – confirmado de forma consistente e repetida.

Achado adicional: ao contrário do ESP-IDF nativo, que já possui internamente proteção contra reaplicação automática de uma versão previamente inválida, a implementação própria do cliente OTA sobre o protocolo do ThingsBoard não tinha esta proteção. Consequência observada: como o pacote defeituoso permanecia atribuído ao dispositivo na plataforma, a versão recuperada pelo *rollback* voltava a detetar a mesma atualização pendente a cada reconexão, entrando num **ciclo indefinido de download, falha e re-versão**. Corrigido através da implementação de um mecanismo próprio de memória de falha (persistido em NVS): antes de cada tentativa de atualização, a versão-alvo é registada como "pendente"; se o dispositivo reiniciar sem essa versão se ter tornado a versão em execução, é marcada como "conhecida como inválida", e tentativas futuras dessa mesma versão são recusadas automaticamente, com o estado **FAILED** reportado à plataforma em vez de uma nova tentativa. Reteste após a correção: o dispositivo recuperou corretamente da falha e recusou-se a tentar a mesma versão inválida novamente, estabilizando sem intervenção manual – comportamento agora equivalente ao do ESP-IDF nativo.

5.6 Alarmes e Notificações

Foi configurado um alarme de deteção de desconexão do dispositivo (*device disconnected*), baseado num atributo de conectividade nativo da plataforma (**active**, gerido automaticamente com base num *timeout* de inatividade configurável por dispositivo), e não num mecanismo *Last Will and Testament* (LWT) MQTT customizado – tentativa inicial que não se revelou fiável nesta oferta *Cloud*. O alarme foi ligado ao *Notification Center* da plataforma, testado com sucesso em canal de email: a desconexão física do dispositivo (corte de alimentação) resultou, dentro de segundos, num alarme na consola e num email de alerta automático – o mesmo mecanismo suporta adicionalmente SMS, Slack e Microsoft Teams, sem qualquer alteração ao firmware.

5.7 Provisionamento Automático de Dispositivos

Foi implementado e testado o mecanismo de *device provisioning* do ThingsBoard: em vez de cada dispositivo ser criado manualmente na consola antes de receber a sua credencial (fluxo utilizado nas secções anteriores), o firmware liga-se numa primeira fase com uma credencial genérica, partilhada por todos os dispositivos do mesmo modelo (não uma credencial por unidade), identificando-se com um nome gerado a partir do endereço MAC do dispositivo (garantidamente único por fabrico). A plataforma cria o dispositivo automaticamente e devolve uma credencial individual, que o firmware guarda em memória não-volátil (NVS) para reutilização em arranques subsequentes. Testado com sucesso: o dispositivo registou-se autonomamente na consola, sem qualquer criação manual prévia – capacidade sem equivalente direto nas duas plataformas anteriores, relevante para um cenário de produção em escala, onde não é viável a criação manual de cada unidade.

5.8 Aplicação Móvel

Confirmado o acesso ao dispositivo e ao *dashboard* através da aplicação móvel oficial do ThingsBoard, sem qualquer configuração adicional – reflete automaticamente o estado da consola web.

5.9 Esforço de Integração

Registo sem cartão de crédito, criação do dispositivo/token em poucos passos, e adaptação de um exemplo MQTT já existente do ESP-IDF (cerca de 15 linhas de código para a telemetria inicial, crescendo com cada funcionalidade adicional testada). Claramente inferior ao esforço estimado para a AWS IoT Core (certificados X.509, políticas IAM, SDK próprio mais pesado), com base na documentação consultada.

5.10 Segurança do Transporte (MQTTS/TLS)

Os testes iniciais foram realizados sobre `mqtt://` (porta 1883), sem encriptação – o *access token* e todos os dados circulavam em texto simples pela rede, lacuna identificada e registada como prioridade de trabalho futuro. Foi posteriormente corrigido: o transporte foi migrado para `mqstts://` (porta 8883), com verificação do certificado do servidor através do *certificate bundle* de autoridades certificadoras já embutido no ESP-IDF (o mesmo mecanismo usado para HTTPS/OTA no ESP-IDF nativo), sem necessidade de embutir manualmente um certificado específico do ThingsBoard – ao contrário do esquema de certificados X.509 por dispositivo, mais pesado, tipicamente usado pela AWS IoT Core.

Resultado: migração bem-sucedida, sem alteração de comportamento visível (provisionamento e ligação normal continuam a funcionar de forma idêntica), confirmada por validação de certificado nos registos série em ambas as ligações MQTT do dispositivo (provisionamento e operação normal). Custo: o binário cresce de aproximadamente 917KB para 987KB devido ao conjunto de autoridades certificadoras incluído, ainda confortavelmente dentro do limite da partição de 1MB.

5.11 Prova de Conceito: Cliente Nativo Zephyr

Ao contrário do ESP RainMaker (SDK construído sobre o ESP-IDF, não pode correr sobre Zephyr – secção 11), o ThingsBoard não está preso a nenhum SDK de firmware:

só exige um cliente MQTT a falar a sua convenção de tópicos. Para confirmar isto na prática e não apenas por análise de arquitetura, foi escrito um cliente MQTT completo sobre o subsistema nativo do Zephyr (`subsys/net/lib/mqtt`), replicando o subconjunto essencial já validado no ESP-IDF: provisionamento automático de dispositivo, ligação MQTTS (TLS) com o mesmo mecanismo de autenticação (`username = access token`), telemetria periódica e RPC (`getStatus/reboot`).

Dois problemas reais encontrados e corrigidos (nenhum deles óbvio a partir da documentação):

- A cadeia de certificados do *broker* `mqtt.eu.thingsboard.cloud` é inteiramente baseada em curvas elípticas (*Sectigo/USERTrust ECC*, P-256 e P-384). A configuração *default* do mbedTLS no Zephyr não tem suporte a curvas elípticas ativo de todo (`CONFIG_MBEDTLS_ECP_C` não estava definido) – ao contrário do ESP-IDF, que traz um *bundle* de autoridades e algoritmos já preparado. Corrigido ativando explicitamente `MBEDTLS_ECP_C`, `MBEDTLS_ECDH_C`, `MBEDTLS_ECDSA_C` e as curvas P-256/P-384.
- As assinaturas da cadeia usam SHA-384 (`ecdsa-with-SHA384`, `sha384WithRSAEncryption`), também ausente da configuração *default* (`CONFIG_MBEDTLS_SHA384` não estava definido) – causa direta do erro inicial (`MBEDTLS_ERR_X509_UNKNOWN_SIG_ALG`, análogo ao caso do Zephyr vs. ESP-IDF em drivers: o Zephyr exige mais configuração explícita onde o ESP-IDF já vem pronto).

Resultado após correção, validado com hardware real: *handshake* TLS bem-sucedido, provisionamento automático concluído com um token real emitido pelo ThingsBoard, e telemetria publicada com sucesso de forma sustentada (múltiplos ciclos da rampa de temperatura sintética 20-29, sem falhas nem desconexões).

Limitação observada, não um defeito: o *token* de acesso não é persistido em memória não-volátil nesta prova de conceito – cada reinício repete o provisionamento. Numa tentativa de reprovisionar o mesmo nome de dispositivo pouco depois de um provisionamento bem-sucedido, o ThingsBoard recusou o pedido ("Failed to provision device!") – comportamento esperado da política de provisionamento do lado do servidor (não um bug do cliente), mas confirma que uma implementação real precisaria de persistir o *token* (ex: subsistema `settings` do Zephyr) em vez de reprovisionar a cada arranque.

Implicação: confirma-se na prática, não só em teoria, que **Zephyr + ThingsBoard** é uma combinação viável – resolvendo a tensão entre o suporte nativo a LoRa do Zephyr e a dependência de ESP-IDF do RainMaker (secção 11).

6 Plataforma de Gestão 2: ESP RainMaker

A avaliação desta plataforma manteve-se a um nível mais geral, sem aprofundar detalhes técnicos específicos como nas secções anteriores. Foi escolhido o ESP RainMaker por ser a solução de *device management* da própria Espressif, fabricante do ESP32, um candidato natural por ser feita à medida deste hardware.

6.1 Esforço de Integração Inicial

Utilizado o exemplo oficial `temperature_sensor` do repositório `espressif/esp-rainmaker`, gerido pelo *IDF Component Manager* moderno (resolução automática de dependências no

primeiro *build*, sem necessidade de `git submodules`). A compilação (`idf.py build`) funcionou **sem qualquer configuração manual** – o esforço de integração inicial mais baixo de todas as plataformas avaliadas neste relatório.

Nota sobre bloqueio de fornecedor (*vendor lock-in*): ao contrário do ThingsBoard e da Azure (protocolos abertos, compatíveis com qualquer fabricante de microcontrolador), o RainMaker é um SDK proprietário da própria Espressif, funcional apenas em chips ESP32 e variantes. Todo o esforço de integração mais baixo já descrito tem esta contrapartida: adotar o RainMaker compromete a Watgrid ao fornecedor Espressif para qualquer sensor futuro, sem a flexibilidade de diversificar hardware que o Zephyr (RTOS multi-fabricante, secção 3) ou as plataformas de *device management* baseadas em protocolo aberto oferecem.

6.2 Arquitetura de Provisionamento – Diferença Relevante

Ao contrário de todas as plataformas anteriores (que assumem credenciais Wi-Fi fixas em tempo de compilação), o ESP RainMaker gera um código QR com uma credencial aleatória a cada arranque, para um fluxo de provisionamento via *Bluetooth* (BLE) ou SoftAP, tipicamente mediado pela aplicação móvel oficial. Esta arquitetura assume um "instalador" humano com telemóvel no momento da implantação, em vez de credenciais pré-programadas na fábrica – uma filosofia de produto diferente da do ThingsBoard.

Tentativa de provisionamento apenas por portátil (sem telemóvel): foi explorada a `esp-rainmaker-cli` (ferramenta Python oficial) como alternativa à aplicação móvel. A via BLE falhou de forma persistente por incompatibilidade entre a biblioteca cliente e a pilha Bluetooth do sistema operativo; a via SoftAP (reconfigurada via Kconfig) conseguiu estabelecer ligação, mas o passo de obtenção de credenciais da cloud ("*claim*") foi recusado pela aplicação oficial com o erro "*Assisted Claiming not supported for SoftAP*". Inspeção do código-fonte do componente `esp_rainmaker` confirmou tratar-se de uma limitação estrutural da aplicação oficial (a capacidade de *claim* é anunciada de forma incondicional, independente da configuração escolhida), não um erro de configuração corrigível. Revertida a configuração para BLE (combinação oficialmente suportada), o fluxo completou-se sem problemas através da aplicação móvel.

6.3 Resultado: Acompanhamento via Consola Web Confirmado

Confirmado que, após o passo único de provisionamento inicial (que exige mesmo a aplicação móvel), o dispositivo fica acessível integralmente através da consola web (`dashboard.rainmaker.esp`) com os mesmos dados/parâmetros visíveis na aplicação – não é necessário voltar a usar o telemóvel para o acompanhamento contínuo.

6.4 Atualização Remota de Firmware (OTA)

O suporte a OTA já vem ativo por omissão no exemplo utilizado (`esp_rmaker_ota_enable_default()`), sem necessidade de código adicional. Testado via consola web (*upload* de imagem em **Firmware Images**, seguido de **Start OTA** associado ao nó).

Erro inicial e correção: a primeira tentativa foi rejeitada pelo dispositivo ("*Current running version is same as the new. We will not continue the update*"), apesar de o metadado de versão na consola ter sido definido como diferente – a versão efetivamente comparada pelo firmware provém de `set(PROJECT_VER "1.0")` no `CMakeLists.txt`, não

do campo preenchido na interface web. Este é o mesmo padrão de erro já cometido nas secções anteriores (ThingsBoard, ESP-IDF nativo): a versão que o dispositivo compara tem de estar embutida no próprio binário, não bastando um metadado externo – reforça tratar-se de uma lição geral, válida para qualquer plataforma de OTA, não uma particularidade de uma só. Corrigida a constante e recompilado, a atualização completou-se com sucesso (transferência de aproximadamente 1.5MB via HTTPS/TLS, escrita da partição, reinício, e confirmação de versão nova nos registos série).

Achado relevante – rollback incluído por omissão: os registos do arranque seguinte ao OTA mostram automaticamente *"First Boot after an OTA"* seguido de *"Diagnostics completed successfully! Continuing execution..."* – o RainMaker gere sozinho o ciclo de arranque pendente/confirmação, sem que tenha sido necessário escrever qualquer código de confirmação manual, ao contrário do ThingsBoard e mesmo do ESP-IDF nativo puro (onde a chamada a `esp_ota_mark_app_valid_cancel_rollback()` teve de ser adicionada explicitamente). O cenário de falha real (imagem defeituosa a reverter) não foi testado, por se manter fora do nível "geral" pedido para esta plataforma, mas a presença deste mecanismo por omissão sugere que o *rollback* é tratado como parte central da experiência *out-of-the-box* do RainMaker.

6.5 Comandos Remotos (RPC)

Os registos de arranque confirmam um módulo de comando/resposta nativo ativo por omissão (*"Enabling Command-Response Module"*, com um *handler* padrão de *"set params"* já registado), visível também como o atributo `cmd-resp: 1` do nó via CLI.

Comando customizado testado com sucesso: implementado um comando `getStatus` (via `esp_rmaker_cmd_register()`, devolvendo temperatura e memória livre em JSON), mesmo espírito do equivalente já validado no ThingsBoard. Testado via `esp-rainmaker-cli` (`create_cmd_request` seguido de `get_cmd_requests`): resposta recebida em aproximadamente 1 segundo, com o conteúdo correto (`{"freeHeap": 153316, "temperature": 26}`). Confirma o RPC customizado a funcionar de ponta a ponta, equivalente ao ThingsBoard.

Nota lateral – credenciais Wi-Fi por dispositivo, não por firmware: ao contrário de todas as outras plataformas testadas (credenciais fixas no binário compilado), o RainMaker guarda a rede Wi-Fi apenas na placa, atribuída em *runtime* durante o provisionamento pela aplicação móvel. Mudar de rede (por exemplo, ao mudar de local de trabalho) exige repetir fisicamente o provisionamento, não uma simples recompilação – um *trade-off* relevante entre conveniência para o utilizador final (a mesma *imagem* de firmware serve qualquer rede) e agilidade durante o desenvolvimento/testes.

6.6 Funcionalidades de Gestão de Frota (levantamento geral)

Confirmada a existência de: **Node Groups** (agrupamento de dispositivos para gestão conjunta – conceptualmente equivalente à combinação de Device Profiles e Customers do ThingsBoard, mas unificado num único conceito), **Insights** (painel de métricas/analytics, equivalente ao já observado no ThingsBoard), e **Local Control** (achado não planeado: um servidor HTTPS local arrancado automaticamente na placa para comunicação direta na rede local sem passar pela cloud, não observado de forma equivalente no ThingsBoard).

Os **Node Groups** não foram testados na prática (o PoC utilizou apenas um dispositivo), mas é uma funcionalidade que se assume relevante para o caso de uso da Watgrid:

com vários sensores distribuídos por uma adega ou vinha, permitiria atualizar todos de uma só vez (OTA, configuração) em vez de um a um – essencial para uma implantação a escalar além de um único dispositivo de teste.

O ThingsBoard cobre o mesmo objetivo através de dois conceitos separados: **Device Profiles** (configuração/OTA partilhada por todos os dispositivos do mesmo tipo – já testado e confirmado a funcionar neste relatório, secção 5.4) e **Customers** (isolamento de dispositivos por cliente final, relevante caso a Watgrid venha a vender a solução a várias adegas distintas). Para o cenário de "atualizar vários sensores na mesma adega de uma vez", o Device Profile do ThingsBoard tem a vantagem de já estar validado na prática, enquanto o Node Groups do RainMaker permanece apenas confirmado conceptualmente.

6.7 Achado Espontâneo: Notificação de Desconexão Sem Configuração

Ao desligar a alimentação da placa fora do contexto de um teste deliberado, foi recebida uma notificação *push* na aplicação móvel a assinalar a desconexão do sensor, sem que tivesse sido configurado explicitamente qualquer alarme ou regra de notificação – ao contrário do ThingsBoard, onde o mesmo resultado exigiu criar manualmente uma *Alarm Rule*, configurar o *Notification Center* e um *template* de email. Reforça o mesmo padrão já observado no *rollback* de OTA: o RainMaker trata deteção de falhas e alertas como parte da experiência incluída por omissão, em contraste com a abordagem mais configurável, mas mais trabalhosa, do ThingsBoard.

6.8 Reavaliação com Foco em Gestão do Dispositivo

Numa segunda fase da avaliação (ver secção 7.7, "Reorientação de Foco"), o critério de avaliação foi redirecionado para a gestão do próprio microcontrolador – reiniciar remotamente, saber se está atualizado, saber se o sinal Wi-Fi é fraco – em vez do processamento de dados de sensores. O RainMaker foi por isso reaberto para testar especificamente estes dois critérios, que tinham ficado por explorar na avaliação "geral" original.

Reboot remoto – nativo, zero código. O SDK do RainMaker inclui um **System Service** pronto a usar (`esp_rmaker_system_service_enable()`), com um parâmetro Reboot do tipo *standard* (`esp.param.reboot`) reconhecido pela aplicação móvel e pela consola web. Ativado com uma única chamada de função (sem escrever qualquer *handler* de escrita – o SDK já contém o código que interpreta este parâmetro e chama `esp_rmaker_reboot()` internamente, confirmado por inspeção direta do código-fonte do componente). Testado com sucesso de duas formas: via `esp-rainmaker-cli setparams {"System":{"Reboot":true}}` e diretamente no separador *Config/Params* da consola web – em ambos os casos, a placa reiniciou fisicamente, confirmado nos registos série pela sequência de arranque fresca e pelo valor do parâmetro a repor automaticamente para `false` após a ação. Contraste direto com o ThingsBoard e a Azure, onde o "reboot" foi sempre um RPC/Direct Method *customizado*, cujo significado tivemos de definir e implementar nós próprios.

RSSI e saúde de memória – nativo, via Insights. Ativado o componente **ESP Insights** (já parcialmente integrado no exemplo, apenas desativado por omissão via *Kconfig*). Sem qualquer código adicional, o dispositivo passou a reportar automaticamente: força de sinal Wi-Fi (RSSI atual e mínimo histórico), estado de ligação, e métricas de memória (*heap* livre, maior bloco livre, mínimo histórico, falhas de alocação). Confirmado

tanto nos registos série (`wifi_metrics: Wi-Fi RSSI crossed threshold...`) como visualmente no painel **Insights** da consola (gráficos de série temporal para RSSI e *heap*, por nó). Contraste direto com a Azure, onde o equivalente (`wifiRssi` no Device Twin) exigiu escrever a chamada a `esp_wifi_sta_get_ap_info()` e construir manualmente o relatório – aqui, o mesmo dado (e mais, incluindo saúde de memória) vem incluído, com painel gráfico próprio, sem qualquer linha de código nossa além de ativar a opção.

Achado relevante – reprovisionamento após reutilização do hardware. Ao tentar reprovisionar esta placa (já utilizada nos testes de Azure DPS, secção 7), o fluxo BLE via aplicação móvel falhou repetidamente no passo "*Confirming Node Association*", com um erro genérico (*ESPSessionError error 0*), mesmo após reiniciar a aplicação, alternar Bluetooth, e reiniciar a placa. A causa raiz identificada: dados de associação de um provisionamento anterior (de uma sessão de testes distinta, semanas antes) permaneciam na partição `fctry` da flash, nunca apagada pelas limpezas de NVS feitas durante os testes de Azure (que usam um esquema de partições diferente, com offsets distintos). Resolvido com uma limpeza completa da flash (`esptool.py erase_flash`), após o que o reprovisionamento via aplicação móvel completou-se sem problemas. Achado relevante para qualquer cenário de reutilização de hardware entre plataformas/testes distintos – dados de sessões anteriores podem persistir de forma não óbvia em partições dedicadas, não apenas na NVS genérica.

Ver versão de firmware – nativo, sem código. Confirmado que a versão de firmware (lida automaticamente de `PROJECT_VER` no `CMakeLists.txt`) é exposta como metadado nativo do nó, visível tanto na consola web ("*About this node*") como na aplicação móvel ("*Node Information-> Firmware version*"), sem qualquer código ou configuração adicional da nossa parte.

Agir sobre conectividade – nativo, testado com sucesso (*Wi-Fi Reset*). Usando o mesmo *System Service*, ativado também o parâmetro *standard* `Wi-Fi-Reset` (`esp_rmaker_wifi_reset()`), que limpa apenas as credenciais Wi-Fi do dispositivo, mantendo intacta a associação à conta. Disparado com sucesso via `esp-rainmaker-cli setparams ({"System":{"Wi-Fi-Reset":true}})`: a placa voltou de imediato ao modo de *provisioning* Bluetooth (confirmado nos registos série), sem necessidade de qualquer intervenção adicional para provar a ação – a prova de "agir sobre conectividade" está no simples facto de o dispositivo passar de ligado a modo de configuração por comando remoto. Nota: uma tentativa subsequente de reprovisionamento via aplicação móvel voltou a falhar com o mesmo erro genérico da secção anterior, resolvida de novo com uma limpeza completa da flash – sugerindo que a causa raiz pode ser, pelo menos em parte, uma instabilidade intermitente do lado da aplicação/*cloud* da Espressif, e não exclusivamente dados residuais na placa.

Síntese visual – um único ecrã nativo. A aplicação móvel resume, num único ecrã ("*Node Details*"), praticamente todos os critérios de gestão do dispositivo testados nesta reavaliação: tipo de dispositivo, versão de firmware, e uma secção "*System Services*" com botões diretos para `Reboot` e `Wi-Fi-Reset`, além de um botão nativo adicional não testado explicitamente antes – "*Check firmware update*" (pedido manual de verificação de atualização, disparado pelo utilizador). Contraste direto com o ThingsBoard e a Azure: o mesmo conjunto de ações, nessas plataformas, exigiria construir uma interface própria ou usar chamadas API/CLI com JSON – aqui, são botões de toque simples, nativos da aplicação, sem qualquer desenvolvimento de interface da nossa parte.

Histórico de *reboot/crash* – nativo, via Insights. O painel **Node Report** dos Insights regista automaticamente cada reinício do dispositivo, com a causa exata

(ESP_RST_POWERON para ligação normal, ESP_RST_SW para reinício por software) e talão temporal – confirmados, nos testes desta secção, os dois reinícios remotos disparados (*Reboot* via CLI e via consola web) como duas entradas distintas "Software reset via esp_restart (ESP_RST_SW)". Nenhum código nosso foi necessário para este registo – é parte da funcionalidade de diagnóstico incluída por omissão no componente Insights, complementando o *dashboard* de RSSI/*heap* já descrito.

6.9 Síntese da Comparação com o ThingsBoard

O RainMaker vence no esforço de integração inicial do firmware (*zero-config*) e no *roll-back* de OTA incluído por omissão, sem necessidade de código adicional. O ThingsBoard vence claramente na flexibilidade do fluxo de registo – inteiramente possível via consola/API/CLI, sem dependência de aplicação móvel ou Bluetooth. Após o registo inicial, ambas as plataformas oferecem acompanhamento completo via web, incluindo OTA e comandos remotos.

7 Plataforma de Gestão 3: Azure IoT Hub + Device Provisioning Service (DPS)

Ao contrário do ThingsBoard e do ESP RainMaker, o Azure IoT Hub tinha sido avaliado no corpo original deste relatório apenas por documentação (secção 5), dado o esforço de integração aparentemente maior (certificados X.509, políticas IAM/RBAC) e a exigência de cartão de crédito para criar conta. Esta secção documenta uma avaliação prática (PoC) subsequente, focada especificamente no mecanismo de *device provisioning* da Azure – o **Device Provisioning Service (DPS)** –, item já identificado como trabalho futuro.

7.1 Configuração dos Recursos na Azure

Foram criados, numa subscrição gratuita *Azure for Students* (sem necessidade de cartão de crédito adicional, por já estar associada a e-mail institucional), dois recursos no mesmo *resource group*: uma instância de DPS (*watgrid-dps*) e um IoT Hub (*watgrid-hub*, *tier Free*, limite de 8.000 mensagens/dia), ligados entre si via **Linked IoT hubs**.

Restrição de região encontrada: a criação da instância DPS foi inicialmente recusada pela Azure (*"RequestDisallowedByAzure"*) nas regiões **East US** e **West Europe**, por política antifraude específica de subscrições *Students*/gratuitas, que restringe os recursos a um subconjunto de regiões não documentado antecipadamente na interface. Resolvido por tentativa direta de outra região (**France Central**), sem impacto na validade do teste – mas um obstáculo de integração adicional, não mencionado na avaliação por documentação original.

7.2 Arquitetura de Provisionamento

O DPS resolve o mesmo problema já validado no ThingsBoard (secção 5.7) – um dispositivo de fábrica, sem identidade individual, obtém autonomamente uma credencial própria no primeiro arranque –, mas divide-o estruturalmente em dois serviços separados: o **DPS** decide a que IoT Hub um dispositivo deve ser atribuído (relevante a grande escala, para balancear dispositivos por múltiplos Hubs/regiões), enquanto o **IoT Hub** é o destino onde

o dispositivo efetivamente opera depois. Foi configurado um *Individual Enrollment* com mecanismo de *attestation Symmetric Key* (mais próximo, em esforço, do *access token* do ThingsBoard do que do esquema X.509 completo), associando um **Registration ID** (**esp32-winegrid**) a uma chave partilhada (*Primary Key*) e ao Hub de destino.

Ao contrário do *access token* estático do ThingsBoard (copiado diretamente para o firmware), a autenticação do dispositivo na Azure – tanto no DPS como, depois, no próprio IoT Hub – assenta num **SAS token** calculado localmente: uma assinatura HMAC-SHA256 sobre o recurso-alvo e um instante de expiração, gerada com a *Primary Key* (nunca transmitida diretamente pela rede). Esta dependência de um instante de expiração absoluto introduziu um requisito não presente nas plataformas anteriores: sincronização de relógio via **SNTP** logo após a ligação Wi-Fi – sem isto, o token gerado teria uma data de expiração inválida (o ESP32 arranca à época Unix de 1970) e seria rejeitado pela Azure.

7.3 Esforço de Integração (Firmware)

Por coerência com a abordagem já validada no ThingsBoard (**esp-mqtt** direto, sem SDK pesado da plataforma), a integração com a Azure foi feita da mesma forma – implementação manual do protocolo MQTT do DPS e da geração de SAS tokens (via **mbedtls**), em vez do componente oficial **esp-azure** (*port* do **azure-iot-sdk-c**, com dependências e tempo de compilação consideravelmente maiores). Isto exigiu significativamente mais código do que o equivalente do ThingsBoard: a autenticação por assinatura local (*vs.* token estático) e o protocolo de registo do DPS por *polling* (pedido inicial, resposta intermédia de estado "assigning", novo pedido de estado até resposta terminal "assigned") não têm equivalente direto no fluxo de provisionamento do ThingsBoard, que se resolve num único pedido/resposta.

7.4 Resultado: Registo e Telemetria Validados

Testado o ciclo completo, com a partição NVS explicitamente apagada para simular uma placa de fábrica: o dispositivo ligou-se ao endpoint global do DPS, recebeu uma resposta intermédia de estado "assigning", voltou a perguntar, e recebeu a atribuição final (**watgrid-hub.azure-devices.net, deviceId=esp32-winegrid**) – confirmado nos registos série. De seguida, o dispositivo ligou-se diretamente a esse Hub com um novo SAS token e publicou telemetria simulada (**temperature**) com sucesso, confirmado tanto nos registos série como na consola Azure (**Devices**, dispositivo **esp32-winegrid** criado automaticamente, sem intervenção manual; e **IoT Hub Usage**, contagem de mensagens a subir de forma consistente com o intervalo de publicação de 10 segundos configurado no firmware).

Nota de reprodutibilidade: a atribuição Hub/*deviceId* fica também guardada em NVS após o primeiro provisionamento (mesmo padrão de cache já usado para o *access token* do ThingsBoard), pelo que arranques subsequentes saltam o DPS e ligam-se diretamente ao Hub – confirmado nos registos como um cenário distinto do registo de fábrica.

Correção de quota: o intervalo de publicação inicial (10 segundos, usado só para acelerar a demonstração) equivalia a 8640 mensagens/dia – sozinho, já ultrapassava a quota de 8000 mensagens/dia do *tier* Free do Hub. Corrigido para 60 segundos por omissão (≈ 1440 mensagens/dia, 18% da quota), valor também mais realista para um

sensor de temperatura numa adega do que um intervalo pensado para demonstração.

7.5 Comandos Síncronos: Direct Methods

Equivalente direto ao RPC já validado no ThingsBoard (secção 5.3) – `getStatus` (devolve temperatura e memória livre) e `reboot`, invocados via consola Azure (**Devices -> esp32-winegrid -> Direct method**). Ao contrário do RPC do ThingsBoard (nome do método dentro do corpo JSON), a Azure identifica o método diretamente no tópico MQTT (`$iothub/methods/POST/{nome}/?$rid={id}`), com resposta obrigatória num tópico espelhado com código de estado (`$iothub/methods/res/{status}/?$rid={id}`).

Resultado: ambos os métodos testados com sucesso – `getStatus` respondeu `status:200` com `{"temperature":26,"freeHeap":184480}` em poucos segundos, confirmado tanto na consola como nos registos série (`Direct method invoked: getStatus`). Testado adicionalmente um método não implementado (`foo`), confirmando o tratamento correto de erro: resposta `status:404` com `{"error":"unknown method"}`, em vez de silêncio ou *timeout*.

7.6 Configuração Remota: Device Twin

Equivalente ao mecanismo de *Shared Attributes* do ThingsBoard (usado para `publishInterval`, secção 5.2), mas com uma distinção estrutural que o ThingsBoard também possui, embora não tenha sido explorada nas secções anteriores deste relatório: o *Device Twin* da Azure separa explicitamente duas metades de um mesmo documento JSON por dispositivo – `properties.desired` (a *cloud* escreve, o dispositivo lê/aplica; equivalente ao *Shared Attribute* do ThingsBoard) e `properties.reported` (o dispositivo escreve sobre si próprio, a *cloud* só lê; equivalente ao *Client Attribute* do ThingsBoard, mecanismo disponível mas nunca utilizado nas secções anteriores – o reporte de `current_fw_version` ao ThingsBoard foi feito via telemetria, não via Client Attribute).

Implementado: ao ligar-se, o dispositivo pede o *twin* completo (`$iothub/twin/GET`), aplica um eventual `desired.publishInterval` já definido, subscreve alterações futuras em tempo real, e reporta de volta (`properties.reported`) a sua versão de firmware e o intervalo efetivamente em uso – com um limite mínimo de 1000ms para evitar valores que voltassem a violar a quota de mensagens.

Resultado: alterado `desired.publishInterval` de 5000 para 50000 diretamente no editor JSON da consola, com o dispositivo já ligado. A alteração chegou ao dispositivo em tempo real (sem reconexão), confirmada nos registos série (`publishInterval updated remotely (device twin) to 50000 ms`), seguida da confirmação em `reported` momentos depois – validando o fluxo bidirecional completo (*desired* → dispositivo aplica → *reported* confirma), incluindo o caso de a alteração ser recebida enquanto o dispositivo já está ligado (não só num pedido inicial de arranque).

7.7 Reorientação de Foco: Gestão do Dispositivo, não Telemetria

Nesta fase da avaliação, o critério relevante passou a ser o suporte da plataforma à **gestão do próprio microcontrolador** – saber se está atualizado, se a ligação Wi-Fi é fiável, o estado geral do hardware – e não o processamento dos *dados* que os sensores produzem. As duas subsecções seguintes (*Message Routing* e, em menor grau, C2D) situam-se mais do lado dos *dados* do que do dispositivo em si, e devem ser lidas como exploração lateral, não como critério central desta avaliação.

Em resposta direta a este critério, foram acrescentadas ao `properties.reported` do Device Twin (já implementado na secção anterior) duas métricas de saúde do dispositivo: a força do sinal Wi-Fi (`wifiRssi`, em dBm, obtida via `esp_wifi_sta_get_ap_info()`) e a memória livre (`freeHeap`). Confirmado nos registos série: `{..., "wifiRssi": -63, "freeHeap": 178988}` coerente com o valor reportado pelo próprio driver Wi-Fi na ligação.

Visibilidade de frota via Queries: explorada a funcionalidade **Device management** → **Queries** do IoT Hub – uma consulta SQL-like sobre os *twins* de todos os dispositivos em simultâneo, sem abrir cada um individualmente. A consulta `SELECT deviceId, properties.reported.current_fw_version, properties.reported.wifiRssi, properties.repor` FROM `devices` devolveu, numa única chamada, o estado dos três dispositivos já criados neste PoC: `esp32-winegrid-groupA` (versão 1.0, sinal -62dBm), `esp32-winegrid-fake` (sem propriedades, como esperado, por nunca ter hardware associado), e `esp32-winegrid` (versão 1.1, mas `fw_state: "FAILED"` – resíduo do teste de *rollback* da secção anterior, sinalizando de imediato, sem qualquer inspeção manual, que este dispositivo teve uma tentativa de atualização recusada). Esta é exatamente a visibilidade de gestão de firmware/conectividade a nível de frota que o critério pretende avaliar – superior à inspeção dispositivo-a-dispositivo já feita nas plataformas anteriores.

7.8 Mensagens Cloud-to-Device (C2D)

Terceiro e último mecanismo de comunicação *cloud*→dispositivo testado nesta secção, distinto tanto dos *Direct Methods* (síncrono, resposta obrigatória dentro de um *timeout*) como do *Device Twin* (estado persistente, "último valor conhecido"): o C2D é uma fila de mensagens pontuais, entregues assim que o dispositivo estiver ligado e subscrito, sem resposta esperada – um modelo *fire-and-forget*. Sobre MQTT, a simples receção da mensagem (PUBLISH com QoS 1) já é suficiente para a remover da fila do lado da Azure, sem necessidade de um *accept/reject/abandon* explícito como exigiria o protocolo AMQP alternativo da Azure.

Implementado como reação de demonstração concreta: qualquer mensagem C2D recebida força uma publicação de telemetria imediata, interrompendo o ciclo normal (60–90s, consoante o valor de `publishInterval` em vigor) – cenário prático de "pedir uma leitura agora" a um sensor na vinha, sem esperar pelo próximo ciclo agendado.

Resultado: enviada uma mensagem ("*Message to Device*" na consola, corpo de texto livre) para o dispositivo já ligado. Entregue e processada em poucos milissegundos, confirmada nos registos série (C2D message received: leitura urgente, seguido de Publishing telemetry immediately (C2D request) e da telemetria publicada fora do ciclo normal).

7.9 Atualização Remota de Firmware (OTA)

O serviço gerido oficial da Azure para OTA (*Device Update for IoT Hub*) está indisponível no *tier* Free do Hub usado neste PoC. Em alternativa, a atualização foi sinalizada através do próprio *Device Twin* já validado – `desired_fw_version` e `desired_fw_url` –, reaproveitando o mecanismo de download HTTP e o esquema de partições duplas já validados no ESP-IDF nativo (secção 4), servido por um servidor HTTP local tal como nessa secção. O progresso é reportado de volta via `properties.reported_fw_state`, com a mesma máquina de estados (DOWNLOADING → VERIFIED → UPDATED) já usada no cliente OTA do ThingsBoard.

Resultado: atualização bem-sucedida. Definido `desired.fw_version="1.1"` com o respetivo `fw_url`, o dispositivo (a correr a versão 1.0) descarregou a imagem, escreveu-a na partição OTA seguinte, reiniciou, e arrancou a partir dessa partição (confirmado nos registos de arranque: `Loaded app from partition at offset 0x110000`, a partição `ota_0`, não a `factory`). A confirmação de *rollback* (reutilizando `esp_ota_mark_app_valid_cancel` já validada no ESP-IDF nativo) e o reporte da nova versão em `reported.current_fw_version` completaram o ciclo com sucesso.

Resultado: *rollback* e proteção anti-ciclo – com um bug real encontrado e corrigido durante o teste. Repetido o teste de falha real já aplicado ao ThingsBoard e ao ESP-IDF nativo: publicada uma imagem 1.2 com uma chamada a `abort()` no arranque, sinalizada via `desired.fw_version`. Na primeira tentativa, o dispositivo entrou num **ciclo indefinido de download, *crash* e reversão** – o mesmo padrão de falha já documentado para o ThingsBoard (secção 5.5), mas desta vez causado por um bug próprio: a função de reconciliação pós-arranque (que compara a versão pendente em NVS com a versão a correr, para detetar reversões) executava *antes* da chamada a `abort()` de teste, pelo que confirmava erradamente a atualização como bem-sucedida (por coincidência de versão) no mesmo arranque em que a imagem ainda iria falhar – apagando a marca de "pendente" antes de esta poder ser usada para detetar a reversão no arranque seguinte. Corrigido ao mover o `abort()` de teste para antes da reconciliação, replicando fielmente a ordem real de uma falha em produção (a aplicação falha antes de ter oportunidade de reconciliar o arranque anterior). Reteste após a correção: o dispositivo descarregou a imagem defeituosa, reiniciou, falhou, o *bootloader* reverteu automaticamente para a última versão estável, e a reconciliação seguinte identificou corretamente a versão 1.2 como inválida (`Firmware 1.2 previously failed on this device - refusing to retry automatically`) – confirmado nos registos série numa tentativa de reativação subsequente (via `reboot`), que foi recusada automaticamente sem novo download, estabilizando sem intervenção manual, tal como o comportamento equivalente já validado no ThingsBoard.

7.10 Alarmes/Notificações: Azure Monitor (explorado, não validado)

Equivalente ao alarme de desconexão já configurado e testado no ThingsBoard (secção 5.6): uma **Alert Rule** do Azure Monitor sobre o `watgrid-hub`, com a métrica nativa *"Connected devices"* (*Average* < 1, avaliada a cada minuto, janela de 5 minutos) e uma *Action Group* a enviar email. Ao contrário de todos os outros recursos usados neste PoC (DPS, IoT Hub *tier* Free, mensagens dentro da quota gratuita), a consola indicou um custo explícito para esta regra especificamente (\$0.10 USD/mês, cobrado ao crédito gratuito da subscrição *Azure for Students*, não a um cartão de crédito).

Optou-se por configurar a regra até ao ecrã de condições (confirmando que o sinal *"Connected devices"* está disponível e configurável exatamente como descrito) sem a chegar a criar/ativar, para não incorrer nesse custo, ainda que simbólico. Este critério fica, portanto, **documentado mas não validado na prática** – diferentemente de "não testado" por falta de tempo ou capacidade técnica, é uma decisão deliberada de âmbito face a um custo real (mesmo que mínimo) associado especificamente a esta funcionalidade, ao contrário de tudo o resto avaliado nesta secção.

Outros sinais de alerta disponíveis (apenas por documentação da consola, nenhum configurado): para além de *"Connected devices"*, o `watgrid-hub` expõe outras

métricas nativas alertáveis, agrupáveis por categoria –

- **Conectividade:** *Total devices* (contagem de dispositivos registados, independente do estado de ligação) e falhas de autenticação/ligação (ex. SAS token inválido ou expirado).
- **Telemetria:** *Telemetry messages sent* e *Telemetry messages failed* – um alerta sobre mensagens recebidas a cair para zero detetaria um dispositivo *ligado mas silencioso* (bug de firmware, sensor avariado), um cenário de falha que "*Connected devices*" sozinho não apanha; e *Total number of messages used*, que permitiria alertar *antes* de se atingir a quota diária, em vez de só depois (ao contrário do painel *IoT Hub Usage* já usado neste relatório, que é apenas informativo).
- **Direct Methods e Device Twin:** contagem de invocações de *Direct Methods* falhadas, e volume de leituras/escritas ao *twin*.
- **Limitação de débito:** *Throttling errors*, relevante à medida que o número de dispositivos escala.
- **No próprio DPS** (*watgrid-dps*, recurso separado com métricas próprias): tentativas de registo e atribuições falhadas – mais relevante para deteção de falhas em massa numa fase de produção/fábrica do que no PoC de um único dispositivo aqui descrito.

Para o caso de uso da Watgrid, o sinal mais valioso a seguir ao já configurado seria provavelmente "*Telemetry messages sent*" a cair para zero – mais representativo de uma falha silenciosa realista numa vinha (sensor comprometido, mas ainda ligado à rede) do que uma desconexão total.

7.11 Message Routing (exploração lateral – dados, não dispositivo)

Mecanismo de filtragem de mensagens do lado do Hub, sem custo quando o destino é o *endpoint* embutido gratuito (*events*) – alternativas a um destino próprio (Storage, Service Bus, Event Hub customizado) exigiriam criar um recurso novo. Uma *routing query* avalia cada mensagem recebida (por propriedades de sistema/aplicação, ou pelo próprio corpo) e decide se corresponde a uma regra, encaminhando-a nesse caso ao *endpoint* associado – útil, por exemplo, para separar leituras acima de um limiar crítico do resto da telemetria, sem qualquer alteração ao restante fluxo.

Pré-requisito no firmware: para a Azure conseguir avaliar uma condição sobre o corpo da mensagem (*\$body.temperature*), o dispositivo tem de declarar explicitamente que o conteúdo é JSON – adicionadas as propriedades de sistema *\$.ct=application/json* e *\$.ce=utf-8* ao tópico MQTT de publicação de telemetria (sem isto, o *routing* só teria acesso a propriedades de aplicação/sistema, nunca ao corpo).

Resultado: criada a rota *critical-temperature*, com *data source Device Telemetry Messages, endpoint events* (embutido, gratuito), e *query \$body.temperature > 25*. Validada com a ferramenta de teste sintético da própria consola (corpo de amostra avaliado do lado do servidor, sem envolver o dispositivo real): {"temperature": 30} corresponde à condição, {"temperature": 20} não correspondeu – confirmando a sintaxe e a lógica da *query* nos dois sentidos.

Nota importante sobre o âmbito da validação: o *Message Routing* é avaliado inteiramente do lado do servidor, depois de a mensagem já ter chegado ao Hub – o dispositivo nunca tem, nem pode ter, visibilidade sobre a decisão de encaminhamento (não há nada equivalente nos registos série). A ferramenta de teste da consola usada aqui é também puramente sintética (avalia um corpo de amostra colado manualmente, sem tráfego MQTT real). Uma validação com tráfego real do dispositivo exigiria consultar as métricas do Hub ("*Routing message count*", por *Route name*) ao longo do tempo – não realizada nesta ronda, mas o mecanismo de filtragem em si (a parte que podia estar mal configurada) já está confirmado como correto.

7.12 Gestão de Frota: Configurations + Deployments

Equivalente ao *Device Profile* do ThingsBoard (secção 5, já validado) e aos *Node Groups* do RainMaker (secção 6.6, validado apenas a nível conceptual, com um único dispositivo). O mecanismo de gestão de frota *pós*-provisionamento reside no IoT Hub, em **Configurations + Deployments**: uma configuração que aplica um mesmo *patch* de `properties.desired` a todos os dispositivos cujo *twin* corresponda a uma *query* (por *tag*, por *deviceId*, ou por propriedade *reported*), reavaliada continuamente – inclui automaticamente dispositivos novos que passem a corresponder à condição, sem reconfiguração manual.

Resultado: validado com dois dispositivos, superando o nível conceptual do RainMaker. Dado dispor apenas de um ESP32 físico, foi criado um segundo *device identity* sem hardware associado (`esp32-winegrid-fake`) diretamente na consola, apenas para ter um segundo *twin* na plataforma. Ambos os dispositivos foram marcados com a mesma *tag* (`tags.fleet="vinha-poc"`), e criada uma *Configuration* (`fleet-publish-interval`) com *target condition* `tags.fleet='vinha-poc'` e conteúdo `properties.desired.publishInterval=90000`. O `esp32-winegrid` real aplicou a alteração automaticamente e sem intervenção manual no seu *twin* individual, confirmado nos registos série (`publishInterval updated remotely (device twin) to 90000 ms`); o `esp32-winegrid-fake`, sem hardware, permanece alvejado mas nunca aplicado – comportamento esperado, e ele próprio uma confirmação de que a consola distingue corretamente dispositivos alvejados de dispositivos que efetivamente confirmaram a aplicação.

7.13 Provisionamento em Escala: Group Enrollment

O *Individual Enrollment* usado em toda a secção 7 até aqui assume que cada dispositivo é registado manualmente no DPS antes de existir fisicamente – inviável para produção em fábrica de várias unidades do mesmo modelo (a mesma limitação que motivou o *device provisioning* do ThingsBoard, secção 5.7). O DPS resolve isto com *Group Enrollments*: uma única chave, partilhada por todos os dispositivos do mesmo modelo, da qual cada dispositivo deriva localmente a sua **própria** chave – HMAC-SHA256(`chaveDeGrupo, registrationId`) –, sem que a Azure alguma vez precise de conhecer ou armazenar antecipadamente uma credencial por unidade. A verificação do lado do servidor é simétrica: a Azure guarda apenas a chave de grupo, e recalcula a mesma derivação sempre que um dispositivo se apresenta, confirmando se o resultado bate certo com a assinatura recebida.

Implementação: adicionada ao firmware uma opção de Kconfig (`AZURE_DPS_USE_GROUP_ENROLLMENT`) que, quando ativa, trata o valor configurado como a chave de *grupo* em vez da chave do próprio dispositivo, e deriva localmente a chave efetiva (via `mbedtls_md_hmac`) antes de

qualquer geração de SAS token – reaproveitando integralmente o mecanismo de SAS token já validado para *Individual Enrollment*, que passa a assinar com a chave derivada em vez da chave configurada diretamente.

Resultado: criado um *Group Enrollment* (`esp32-winegrid-group`, *Symmetric Key*, com a `tag fleet="vinha-poc"` aplicada automaticamente a qualquer dispositivo provisionado por este grupo – integrando-se de imediato com a Configuration da secção anterior, sem passo manual adicional). Com a NVS apagada para simular uma placa nova, e uma nova identidade escolhida pelo próprio dispositivo (`esp32-winegrid-groupA`, distinta da `esp32-winegrid` do *Individual Enrollment*), o registo no DPS foi aceite com sucesso usando apenas a chave derivada localmente – confirmado nos registos série (DPS: `assigned to hub=watgrid-hub.azure-devices.net deviceId=esp32-winegrid-groupA`) e na consola, onde ambas as identidades (`esp32-winegrid` e `esp32-winegrid-groupA`) coexistem no mesmo Hub, geradas a partir de dois mecanismos de *attestation* distintos. Prova, com hardware real, que a mesma chave de grupo serve múltiplas identidades simultaneamente, sem qualquer credencial por unidade preparada com antecedência – o cenário de produção em fábrica que o *Individual Enrollment*, por si só, não suporta.

7.14 Limitações do Âmbito deste PoC

O SAS token do dispositivo é gerado com validade fixa de 1 hora, sem renovação automática implementada – adequado para este teste, não para uma implantação de longa duração.

8 Síntese Comparativa

8.1 Mecanismos de OTA (dispositivo)

Critério	MCUboot+mcumgr	LwM2M (Zephyr)	ESP-IDF nativo	ThingsBoard	RainMaker
Transporte	Série	Wi-Fi (CoAP)	Wi-Fi (HTTP)	Wi-Fi (MQTT)	Wi-Fi (HTTPS)
Modelo	Push	Push	Pull	Pull (por blocos)	Pull
Conclusão da atualização	Sim	Sim (após correção – secção 3.6)	Sim	Sim	Sim
Rollback automático	Não	Não testado	Confirmado	Confirmado (+ proteção anti-loop)	Incluído por omissão (falha real não testada)
Reflete caso de uso real	Não	Sim	Sim	Sim	Sim
Esforço de integração	Alto	Alto	Baixo	Baixo	Muito baixo (<i>zero-config</i>)

8.2 Plataformas de Gestão de Dispositivos (visão mais ampla)

Critério	Lothian (LwM2M)	ESP-IDF nativo	ThingsBoard	ESP RainMaker	Azure DPS + IoT Hub
Hardware suportado	Qualquer (standard aberto OMA)	Só ESP32 (SDK do fabricante)	Qualquer (MQTT, CoAP, HTTP, sem dependência do fabricante)	Só ESP32, Espressif (SDK proprietário do fabricante do chip)	Qualquer (protocolo aberto, SDK multi-plataforma)
Autenticação/criptografia	Não	Não	Sim	Sim	Não testado, exigiria serviço adicional, ex. IoT Central
Configuração remota	Não	Não	Sim (abstrai atributos)	Sim (parâmetros)	Sim, confirmado (deveria ser <i>desired-report</i>)
Comunicação assíncrona (RPC)	Não	Não	Sim, confirmado	Sim, confirmado	Sim, confirmado (deveria ser <i>desired-report</i>)
OTA gerido pelo cliente (conexão cloud/secure)	Não (até hoje)	Não (serviço HTTP próprio)	Sim, confirmado	Sim, confirmado	Parcial (conexão via <i>device twin</i> sem conexão dedicada. Deveria: <i>Update independent no tier Free</i>)
Rollback de OTA	Não testado	Confirmado (implementado à mão)	Confirmado (implementado à mão)	Confirmado (incluído por omissão)	Confirmado (implementado à mão, com proteção anti-círculo viciado durante o teste)
Atenuação de conflitos	Não	Não	Sim (ruído, SDK etc., configurado manualmente)	Sim (certificação <i>push</i> nativa, alterada espontaneamente, sem configuração)	Excluído, não atenuado (casos mistos explicados no contexto, mesmo que mínimos)
Provisionamento automático	Não	Não	Sim (comando API, CLI)	Sim (app, BLE, sistema)	Sim, confirmado (DPS, sem app, BLE, individual e group enrollment)
Gestão de frotas (configurar dispositivos)	Não	Não	Sim	Sim (UI de <i>Groups</i> , no navegador)	Sim, confirmado (configuração → <i>Deployment</i> 2 dispositivos)
Aplicação nativa	Não	Não	Sim (opcional)	Sim (necessária no registo)	Não
Esforço de integração inicial (hardware)	Alto	Baixo	Baixo	Muito baixo (<i>zero-config</i>)	Alto (SAS tokens e protocolo do DPS implementados à mão)
Custo	Grátis (<i>esp-idf</i>)	Grátis	Grátis até 5 dispositivos	Grátis até 5 dispositivos	Grátis (tier Free, 1 Hub por subscritor, não <i>upgraded</i>)

8.3 Ver vs. Gerir: Quem Implementa Cada Capacidade

Distinção relevante: para cada capacidade de gestão do dispositivo, não basta perguntar "a plataforma tem isto?-- importa também separar **ver o estado** (leitura) de **agir sobre ele** (escrita/comando), e, para cada um, se a lógica foi implementada **por nós** (usando um canal genérico da plataforma, sem significado embutido) ou já vem **pronta da plataforma** (nativo, sem código nosso). Isto tem impacto direto no critério "Esforço de integração" já usado nas tabelas anteriores – não é o mesmo dizer "a plataforma suporta X" quando X exige 200 linhas nossas ou uma única chamada de função.

Capacidade	ThingsBoard	RainMaker	Azure (DPS+Hub)
Ver versão de firmware	Manualmente (telemetria)	Plataforma (metadado nativo do nó)	Manualmente (<i>reported</i>)
Disparar atualização (OTA)	Híbrido (pacote: plataforma; download: manual)	Plataforma (consola própria, zero código)	Manualmente (Device Twin)
Reiniciar remotamente	Manualmente (RPC customizado)	Plataforma (<i>System Service</i> nativo)	Manualmente (Direct Method customizado)
Ver força de sinal Wi-Fi (RSSI)	Não testado	Plataforma (Insights, zero código)	Manualmente (<i>wifiRssi</i> no <i>twin</i>)
Ver estado de conectividade	Plataforma (active, nativo)	Plataforma (notificação espontânea)	Plataforma (<i>connectionState</i>)
Agir sobre conectividade	Não testado	Plataforma (<i>Wi-Fi Reset</i> nativo)	Não testado
Ver memória/saúde geral	Manualmente (RPC customizado)	Plataforma (Insights, zero código)	Manualmente (<i>twin</i> + Direct Method)

Padrão observado: depois de completada a reavaliação (secção 6.8), o RainMaker é a **única** das três plataformas com todas as 7 capacidades desta tabela nativas – incluindo, agora confirmado, a versão de firmware (metadado automático do nó) e a ação sobre conectividade (*Wi-Fi Reset*, testado com sucesso via CLI, forçando o dispositivo de volta ao modo de *provisioning* sem apagar a associação à conta). O ThingsBoard e a Azure têm apenas 1 capacidade nativa cada (estado de conectividade) do conjunto testado. Isto confirma, com dados concretos e não apenas pela impressão do esforço de integração inicial, que o RainMaker foi desenhado especificamente em torno da gestão do dispositivo em si – exatamente o critério que motivou esta reavaliação. Em contrapartida, ThingsBoard e Azure exigem quase sempre que se defina o significado de cada ação sobre canais genéricos (RPC/*Attributes* num, *Direct Methods*/*Twin* noutro) – o que dá mais controlo/flexibilidade (por exemplo, o formato exato da resposta), ao custo de mais código.

8.4 Integração de Drivers de Sensores I2C e LoRa: Zephyr vs. ESP-IDF

Complementando a comparação anterior, focada nas plataformas de gestão na nuvem, esta síntese cobre especificamente a camada de firmware (secções 3.6 e 4.5), relevante para a escolha do sistema base que vai correr no microcontrolador e comunicar com os sensores em campo.

Critério	Zephyr RTOS	ESP-IDF nativo
Subsistema de <i>drivers</i> de sensores <i>in-tree</i>	Sim (<i>drivers/sensor/</i>)	Não existe
IIS2ICLX (acelerómetro/inclinómetro)	Nativo (I2C e SPI)	Não encontrado
KX122 (acelerómetro)	Não encontrado	Não encontrado
MCP3221 (ADC I2C)	Não encontrado	Não encontrado (mas trivial de implementar)
Suporte nativo a LoRa (transcetores SX126x/SX127x)	Sim (<i>drivers/lorawan/</i> , SX126x/SX127x/RYLR)	Não (depende de componente de terceiros)
Módulos RN2483/WLR089U0 (candidatos concretos)	Não encontrado	Não encontrado
Origem dos <i>drivers</i> em falta	Implementação própria, mesma API do subsistema	Biblioteca de terceiros (Component Registry/GitHub)

Padrão observado: nenhum dos dois *frameworks* tem *drivers* prontos para os três sensores concretos considerados – apenas o IIS2ICLX está coberto, e só no Zephyr. A diferença estrutural está noutro nível: o Zephyr integra sensores e LoRa como parte do próprio sistema operativo, com uma API de *driver* unificada e consistente entre chips (*sensor_sample_fetch/sensor_channel_get* para sensores, *lora_send/lora_recv* para rádio), pelo que integrar um sensor novo – mesmo sem *driver* existente – segue um padrão já estabelecido. O ESP-IDF, por desenho, não tem este subsistema: todo o suporte a sensores e LoRa depende sempre de bibliotecas de terceiros, funcionais mas fora da manutenção direta da Espressif.

Nuance importante para os módulos LoRa concretos considerados (RN2483, WLR089U0): a vantagem do Zephyr em LoRa aplica-se aos transcetores *nus* (SX126x/SX127x) – para estes dois módulos específicos, que comunicam por UART com comandos de texto em vez de SPI direto ao rádio, nenhum *framework* tem suporte pronto (secções 3.6, 4.5), e o RadioLib (ESP-IDF) confirmadamente também não cobre o RN2483. Isto reduz a diferença entre os dois *frameworks* para este componente concreto: em qualquer um, é

preciso escrever um *wrapper* de comandos por UART – tarefa simples (mais próxima do MCP3221 do que de um *driver* de rádio), com bibliotecas de referência já publicadas (incluindo uma versão independente de plataforma) para adaptar.

8.5 Prova de Conceito: Driver MCP3221 (Zephyr vs. ESP-IDF)

Para não deixar a comparação anterior apenas ao nível teórico, foi escrito um driver completo para o MCP3221 (o mais simples dos dois sensores sem suporte nativo) em ambos os *frameworks*, e compilado com sucesso para o `esp32_devkitc_wroom` (Zephyr) e para o alvo `esp32` (ESP-IDF v5.5.1).

No Zephyr, o driver foi escrito como um módulo *out-of-tree* (sem alterar o repositório Zephyr em si), seguindo a mesma API de sensores usada pelo IIS2ICLX (`sample_fetch/channel_get`), com o dispositivo declarado via *devicetree*. No ESP-IDF, foi escrito como um componente próprio sobre o driver nativo de I2C (`esp_driver_i2c`), expondo uma API de funções específica para este sensor (não existe, no ESP-IDF, um equivalente ao subsistema de sensores do Zephyr para encaixar).

Critério	Zephyr (módulo <i>out-of-tree</i>)	ESP-IDF (componente)
Ficheiros escritos	7	3
Lógica do driver (linhas)	113	123 (86 + 37 do <code>.h</code>)
<i>Scaffolding</i> (Kconfig/CMake/ <i>binding</i>)	42 linhas	4 linhas
Total	155 linhas	127 linhas
Compilou à primeira tentativa	Sim	Sim
<i>Warnings</i>	0	0
Encaixa numa API uniforme já existente	Sim	Não (API própria, inventada para este sensor)

Padrão observado: o esforço total é semelhante em número de linhas (o ESP-IDF até fica ligeiramente abaixo), mas distribuído de forma diferente – o Zephyr exige mais *scaffolding* inicial (*devicetree*, Kconfig) mas o driver resultante integra-se na mesma API usada por qualquer outro sensor do sistema; o ESP-IDF tem menos cerimónia inicial, mas o resultado é uma API de funções só deste driver, sem convenção partilhada com outros sensores. Isto confirma, com números reais e não apenas por comparação de arquitetura, que escrever um driver de raiz para um sensor simples como o MCP3221 é uma tarefa de escala semelhante (uma tarde de trabalho) em qualquer um dos dois *frameworks* – a diferença entre eles importa sobretudo quando já existe (ou não) um driver pronto, não quando é preciso escrever um.

Limitação desta prova de conceito: sem o chip MCP3221 fisicamente disponível, ambos os drivers foram validados apenas ao nível da compilação (*devicetree*/Kconfig corretos, chamadas de API corretas) – a lógica de conversão do valor lido ($\text{raw} \times \text{Vref} / 4096$), escrita a partir da folha de dados do fabricante, não foi validada com hardware real.

8.6 Prova de Conceito: Driver RN2483 (LoRa)

Complementando a nota da secção 3.6 sobre o RN2483/WLR089U0, foi escrito e compilado um segundo driver *out-of-tree* para o Zephyr – desta vez para o módulo RN2483, para validar com código real (não só teoricamente) o esforço de integrar um módulo LoRa concreto que comunica por comandos de texto via UART, em vez do SPI direto aos transdutores *nus* já suportados nativamente.

O driver liga-se à mesma API de LoRa do Zephyr (`lora_config()/lora_send()`) usada por qualquer transceptor nativo – ou seja, do ponto de vista de quem o utiliza, um

RN2483 com este driver e um SX1276 nativo são intercambiáveis, sem alterar código de aplicação. Internamente, usa E/S de UART simples por *polling* (`uart_poll_in/uart_poll_out`), não o subsistema mais elaborado de *modem chat* que o Zephyr disponibiliza e que o *driver* nativo do RYLR (`drivers/lora/rylrx.c`) usa – opção deliberada para manter esta prova de conceito no mesmo registo de simplicidade do MCP3221, e não da complexidade de um *driver* de produção.

Resultado: compilou sem erros nem *warnings* à primeira tentativa, para o mesmo alvo (`esp32_devkitc_wroom`). Implementa a inicialização (pedido `sys_get_ver`, para confirmar comunicação com o módulo), configuração (`radio_set_freq/pwr/sf/bw`) e envio (`radio_tx`, com conversão do *payload* para hexadecimal e espera pela confirmação assíncrona `radio_tx_ok`). Total: 302 linhas (270 de lógica do *driver* + 32 de *scaffolding* – *devicetree binding*, Kconfig, módulo Zephyr) – mais do que o MCP3221 (155 linhas), consistente com o RN2483 ter um conjunto de comandos maior (configuração de rádio) do que a leitura trivial de um ADC.

Limitação desta prova de conceito: sem o módulo RN2483 fisicamente disponível, validado apenas ao nível da compilação e da correção da API/*devicetree* – o formato exato dos comandos e respostas foi escrito a partir da documentação oficial da Microchip, mas não testado contra o módulo real.

9 Candidato Adicional: Mongoose OS

Após a síntese comparativa anterior, foi sugerido considerar também o Mongoose OS (Cesanta Software) como candidato de firmware, dada a sua reputação histórica. Foi avaliado com o mesmo rigor prático das restantes camadas de firmware – não só por documentação, mas com instalação, compilação e gravação reais.

9.1 Estado de Manutenção

Investigação do repositório oficial (`cesanta/mongoose-os`) revelou um padrão de manutenção preocupante: o último *release* publicado é a v2.20.0, de dezembro de 2021, e a última alteração de código-fonte real (não documentação) data de fevereiro de 2023 (uma correção de compilação para ESP32-C3). Os *commits* mais recentes visíveis no repositório são exclusivamente adições de *links* para artigos de blog no README, feitos por um único colaborador externo recorrente – não por código novo. Há ainda uma *pull request* de suporte a ESP32-C6 por integrar, e *issues* abertas sobre a própria ferramenta de instalação (*mos*) estar avariada, sem resposta.

9.2 O Que Substituiu o Mongoose OS

A Cesanta Software, a empresa por trás do Mongoose OS, não desapareceu nem encerrou – mas parece ter deixado de investir na sua oferta de **sistema operativo completo**, concentrando-se noutro produto seu, mais modesto em âmbito: a **Mongoose Library** (repositório `cesanta/mongoose`, distinto do `cesanta/mongoose-os` investigado acima).

O que é: uma biblioteca de rede em C – pilha TCP/IP, HTTP, MQTT, *WebSocket*, TLS, e agora também suporte a *rollback* de OTA para várias arquiteturas de microcontrolador. Ao contrário do Mongoose OS, não é um sistema operativo nem exige um *build system* próprio: usa-se como componente **dentro** de outro sistema – bare-metal,

FreeRTOS, e também dentro do próprio Zephyr ou ESP-IDF, como alternativa aos seus subsistemas de rede nativos.

Evidência de manutenção ativa (contraste direto com o Mongoose OS):

- *Releases* regulares e recentes: v7.22 (junho de 2026), v7.21 (abril de 2026), v7.20 (novembro de 2025), v7.19 (agosto de 2025), v7.18 (maio de 2025) – cadência de poucos meses entre versões, sem interrupção.
- *Commits* de código real (não documentação) até à véspera da consulta a este repositório, incluindo funcionalidades novas (ex: *"Add ARCH_MCUXPRESSO and OTA rollback for IMXRT boards"*) e correções de bugs latentes.

Implicação para este relatório: se o objetivo fosse aproveitar código/bibliotecas de rede da Cesanta especificamente (em vez do sistema operativo completo), a Mongoose Library seria a via correta a considerar – não o Mongoose OS. Não foi avaliada em profundidade neste relatório por ter um âmbito bastante mais estreito (só rede, sem gestão de sensores/drivers, sem OTA de aplicação completa, sem *scripting*) do que o que se pretende desta camada de firmware – mas fica registada como referência, caso a Watgrid volte a considerar produtos da Cesanta no futuro.

9.3 Teste Prático: Instalação, Compilação e Gravação

Apesar do diagnóstico de manutenção acima, foi testado se o Mongoose OS ainda é utilizável hoje, com o mesmo hardware (`esp32_devkitc_wroom`) das restantes camadas de firmware:

- **Instalação da ferramenta `mos`:** bem-sucedida via o script oficial (`install.sh`). A verificação de versão mais recente falhou com um erro HTTP 404 (`mos-latest/version.json`) – pequeno sinal adicional de infraestrutura de suporte degradada, mas sem impedir o uso da ferramenta.
- **Compilação:** clonada a aplicação de demonstração oficial (`mongoose-os-apps/demo-c`) e compilada com sucesso para ESP32. **Achado relevante:** o comando `mos build` não compila localmente – envia o código-fonte para um serviço de compilação na nuvem da própria Cesanta, que devolve o binário já compilado. Isto é uma diferença estrutural importante face ao Zephyr e ao ESP-IDF, que compilam inteiramente offline: adotar o Mongoose OS implica uma dependência contínua da infraestrutura de nuvem da Cesanta continuar operacional, um ponto único de falha adicional, agravado pelo contexto de manutenção reduzida.
- **Gravação e arranque:** o binário resultante foi gravado com sucesso via `mos flash`, e o dispositivo arrancou corretamente na primeira tentativa, confirmado pelos registos série (*ticks* de aplicação com contagem de tempo de atividade e memória livre, a cada segundo, de forma estável).

Conclusão: ao contrário do que o estado de manutenção poderia sugerir, o Mongoose OS **continua funcional hoje**, de ponta a ponta (instalação, compilação, gravação, arranque), confirmado com hardware real. A reserva não é "está avariado" – é a falta de evolução (sem suporte a chips novos, sem correções de segurança recentes) e a dependência de um serviço de compilação na nuvem de terceiros que pode deixar de estar disponível sem aviso, dado o baixo nível de investimento atual da própria Cesanta neste produto.

9.4 Drivers de Sensores e LoRa

Não foi encontrado, no ecossistema de bibliotecas oficiais (`mongoose-os-libs`), suporte para o KX122 ou o IIS2ICLX – existe uma biblioteca `imu` com API unificada para acelerómetros/giросcópios, mas cobre cerca de 20 chips diferentes (ADXL345, MPU60x0, ICM20948, entre outros), não estes dois. O MCP3221 também não tem biblioteca I2C dedicada – só uma biblioteca genérica de ADC, que envolve o conversor interno do próprio microcontrolador. Não foi encontrada nenhuma biblioteca de LoRa oficial. O padrão é, portanto, o mesmo já observado no Zephyr e no ESP-IDF (secções 3.6, 4.5): nenhum dos três *frameworks* tem estes componentes específicos prontos.

9.5 Funcionalidades Notáveis (mesmo não adotado)

Vale a pena registar, para referência futura, as capacidades que o Mongoose OS oferece nativamente e que nenhuma das camadas de firmware avaliadas neste relatório tem de forma equivalente:

- **mJS (*scripting* em JavaScript):** permite alterar lógica da aplicação sem recompilar/regravar o firmware completo – nem o Zephyr nem o ESP-IDF oferecem isto (ambos exigem recompilação total em C para qualquer alteração).
- **mDash:** painel de gestão de dispositivos próprio e integrado, incluindo OTA por simples *upload* de um ficheiro (o servidor trata da divisão em blocos automaticamente).
- **Config subsystem:** configuração remota estruturada, nativa do sistema operativo, conceptualmente semelhante ao *Device Twin* do Azure ou às *Shared Attributes* do ThingsBoard, mas sem código adicional necessário.
- **RPC extensível:** lista de serviços RPC que o dispositivo expõe, descobrível remotamente (`RPC.List`) – útil para saber em runtime o que um dispositivo suporta, sem consultar o código-fonte.
- **Provisionamento pronto para múltiplas nuvens:** comandos dedicados para configurar o dispositivo diretamente para AWS IoT, Azure IoT Hub, Google IoT Core e IBM Watson IoT, além do mDash próprio – nenhuma das camadas de firmware avaliadas neste relatório tem este leque de atalhos embutido.
- **Hardware suportado:** ESP32, ESP8266, TI CC3200/CC3220 e STM32 – mais alargado do que o ESP RainMaker (só ESP32/Espressif), embora sem o suporte multi-arquitetura do Zephyr.

Estas funcionalidades confirmam que o Mongoose OS foi, em tempos, um concorrente genuinamente completo – reforçando que a razão para não o adotar agora é exclusivamente o risco de manutenção/futuro, não uma limitação técnica de fundo.

10 Trabalho Futuro

Aprofundar mais critérios de segurança específicos (ex: Secure Boot) não é prioritário nesta fase – o foco deve manter-se na avaliação geral e abrangente de plataformas, antes de afunilar para detalhes técnicos específicos de uma só.

- Validação formal de uma regra de alarme configurada manualmente no ESP RainMaker (até agora só se observou uma notificação nativa espontânea, sem configuração), caso se pretenda considerá-lo a par do ThingsBoard na decisão final.
- Ativar e validar de facto o Alert Rule de alarmes/notificações do Azure Monitor (secção 7), cuja configuração foi explorada mas não criada por implicar um custo mensal explícito, ao contrário de todo o resto avaliado neste relatório. Provisionamento, comandos remotos (*Direct Methods*), configuração remota (*device twin*) e OTA (com *rollback* e proteção anti-ciclo) já foram validados na prática.
- Avaliação prática (PoC) da AWS IoT Core (até agora avaliada apenas por documentação), para manter a abrangência da comparação entre os três grandes fornecedores *cloud*.
- Avaliação da atualização via protocolo HTTP simples sobre Zephyr (terceira arquitetura originalmente identificada), embora de prioridade reduzida face aos resultados obtidos.
- Robustez adicional: comportamento sob perda de energia durante a escrita, testes de carga/*stress* com múltiplos dispositivos.
- **Prioridade decorrente da conclusão (secção 11):** validar a fiabilidade do provisionamento do RainMaker com múltiplas unidades reais e em condições de campo, antes de o adotar como recomendação principal – a instabilidade do fluxo BLE/aplicação móvel, observada repetidamente com um único dispositivo nesta ronda de testes, é a principal reserva à recomendação atual.
- Completar as células "não testado" da tabela *Ver vs. Gerir* (secção 8.3): ação sobre conectividade no ThingsBoard e na Azure, e teste de falha real de OTA no RainMaker (cenário de imagem defeituosa/*rollback*, já validado nas restantes três plataformas).
- Validar com hardware real (MCP3221 físico) os dois drivers de prova de conceito escritos (secção 8.5), confirmando que a leitura/conversão de tensão está correta e não só que compilam.
- Acompanhar o lançamento de uma versão oficial do Zephyr (3.7.x ou posterior) que já incluía a correção da secção 3.5, para deixar de depender de um *cherry-pick* manual do *patch* no checkout local.
- Persistir o *access token* do ThingsBoard em memória não-volátil no cliente Zephyr (secção 5.11), para deixar de reprovisionar a cada arranque – necessário antes de considerar esta combinação para lá de prova de conceito.

11 Conclusão Preliminar

Nota sobre a evolução do critério de avaliação: esta conclusão foi revista a meio da avaliação prática da Azure (secção 7.7), quando o critério de avaliação foi reorientado: o critério relevante para escolher uma plataforma de gestão é o **suporte nativo à gestão do próprio microcontrolador** (firmware, conectividade, saúde do dispositivo) – não o processamento de dados de sensores, que tinha recebido mais peso implícito nas secções iniciais deste relatório. Sob este critério corrigido, e após reabrir o RainMaker e aprofundar a Azure para o testar de forma justa entre as três plataformas (secção 6.8 e tabela "Ver vs. Gerir", secção 8.3), o quadro é mais equilibrado do que a avaliação inicial sugeria.

Segunda nota sobre a evolução do critério de avaliação: esta secção foi revista uma segunda vez, com o foco alargado para incluir a facilidade de integração de *drivers* de sensores I2C e o suporte a comunicação LoRa (secções 3.6, 4.5, 8.4 e 8.5) – critérios relevantes independentemente da plataforma de gestão escolhida, por dizerem respeito à camada de firmware em si. Nesta mesma fase, a falha crítica de OTA sobre Wi-Fi do Zephyr (secção 3.3.2), até aqui tratada como uma limitação não resolvida, foi revisitada e corrigida (secção 3.5) – o que altera a recomendação de firmware abaixo.

No plano do firmware, o **ESP-IDF nativo** continua a ser a opção com menos fricção prática: OTA sem falhas desde o primeiro teste, *rollback* confirmado repetidamente (incluindo em cenários de falha real), esquema de partições robusto, e nenhuma das dificuldades de configuração de *devicetree*/Kconfig observadas no Zephyr. O **Zephyr**, por seu lado, deixa de ter a falha de OTA sem fios como fator eliminatório – identificada a causa raiz (regressão conhecida da versão 3.7.0, com correção oficial já disponível, embora ainda não integrada numa *release* 3.7.x publicada) e confirmada a resolução com um ciclo completo real (680KB via LwM2M/Wi-Fi, sem falhas, *rollback* do MCUboot aplicado com sucesso). Com o requisito de suporte a **LoRa** confirmado como crítico para este projeto, e dado que o Zephyr o suporta nativamente (*drivers/lorawan/*, secção 3.6) enquanto o ESP-IDF depende sempre de bibliotecas de terceiros para o mesmo fim, o Zephyr passa a candidato firme para a camada de firmware, condicionado a: (i) usar um checkout do Zephyr com a correção da secção 3.5 incluída (por *cherry-pick* manual até sair numa *release* oficial, ou por atualização de versão), e (ii) assumir o custo de escrever *drivers* próprios para o KX122 e o MCP3221 – esforço que a prova de conceito da secção 8.5 mostrou ser semelhante ao do ESP-IDF para o mesmo fim (não uma desvantagem exclusiva do Zephyr).

Ao nível da plataforma de gestão, o **ESP RainMaker** é quem melhor cumpre, na prática, o critério agora em foco: das 7 capacidades de gestão do dispositivo testadas de forma equivalente nas três plataformas (versão de firmware, OTA, reiniciar, RSSI, conectividade – ver e agir –, memória), o RainMaker teve todas as 7 nativas, sem uma linha de código nossa, contra 1 do ThingsBoard e 1 da Azure. **Reserva importante:** o fluxo de *provisioning* via Bluetooth/aplicação móvel, único caminho suportado, revelou-se pouco fiável na prática – múltiplas falhas de sessão (*ESPSessionError*) reproduzidas mesmo pela aplicação oficial, nalguns casos só resolvidas com limpeza completa da flash. Para um cenário de implantação real numa vinha/adega, com vários sensores e sem garantia de um técnico com telemóvel disponível de cada vez que for preciso reconfigurar um dispositivo, este é um risco operacional concreto, observado em primeira mão, não hipotético. **Segunda reserva:** o RainMaker é um SDK proprietário, exclusivo a chips ESP32/Espressif – ao contrário do ThingsBoard e da Azure, que aceitam qualquer fabricante de microcontrolador. Adotá-lo compromete a Watgrid a este único fornecedor de hardware, sem a

flexibilidade de diversificar chips no futuro.

O **ThingsBoard** continua a ser a opção mais equilibrada se a fiabilidade e a simplicidade do provisionamento (inteiramente via consola/API, sem dependência de aplicação móvel ou Bluetooth) pesarem mais do que a natividade capacidade-a-capacidade – ao custo de ter de implementar manualmente quase todas as capacidades de gestão do dispositivo (RPC próprio para *reboot*/estado, RSSI nunca testado). Mantém-se a reserva já identificada: o limite de 5 dispositivos do plano gratuito não se aplicaria a uma implantação real (a lacuna de segurança do transporte inicialmente identificada foi entretanto corrigida, MQTTS/TLS, secção 5.10).

A **Azure IoT Hub + DPS**, aprofundada nesta ronda para além da avaliação inicial por documentação, iguala ou supera o ThingsBoard em praticamente todos os critérios técnicos testados – incluindo *Group Enrollment* para provisionamento em escala e *Configurations + Deployments* para gestão de frota, este último mais robusto do que o equivalente ainda só conceptual do RainMaker. O preço é o maior esforço de integração de firmware das três plataformas (sem SDK leve equivalente ao **esp-mqtt**, protocolo do DPS e SAS tokens implementados à mão) e fricção adicional de conta/subscrição (restrição de região, custo explícito nos alarmes). Posiciona-se como o candidato mais forte para escalar além do limite gratuito do ThingsBoard, se a equipa puder absorver esse esforço adicional de engenharia de firmware.

Recomendação: dado o critério corrigido (gestão do dispositivo, não dados), o RainMaker é o candidato tecnicamente mais forte e deve ser o foco de trabalho futuro imediato – mas apenas com uma validação adicional específica da fiabilidade do provisionamento em escala (múltiplas unidades, condições de campo reais), dado o risco concreto já observado nesta secção. Caso essa validação falhe, ou caso se confirme como requisito duro um provisionamento sem dependência de aplicação móvel, a combinação **ESP-IDF nativo + ThingsBoard** permanece a alternativa mais segura e já validada de ponta a ponta, com a Azure como caminho de escala caso o limite de 5 dispositivos do ThingsBoard seja atingido.

Tensão entre as duas camadas – confirmada e validada na prática: o RainMaker é, em si, um SDK construído sobre o ESP-IDF – não corre sobre Zephyr. Se o suporte nativo a LoRa do Zephyr vier a ser tratado como requisito duro ao nível do firmware, isso é incompatível com adotar o RainMaker como plataforma de gestão, e aponta antes para **Zephyr + ThingsBoard** – combinação que deixou de ser apenas teórica: foi escrito e validado com hardware real um cliente MQTT nativo do Zephyr a falar diretamente com a mesma conta ThingsBoard Cloud (secção 5.11), incluindo provisionamento automático, MQTTS e telemetria, sem qualquer SDK de firmware específico exigido. Esta é, por isso, a combinação recomendada caso o LoRa nativo do Zephyr se confirme como requisito duro – em alternativa ao par RainMaker + ESP-IDF, tecnicamente mais forte no critério de gestão do dispositivo mas sem caminho para o Zephyr.